

Next-Latent Prediction Transformers Learn Compact World Models

Jayden Teoh* Manan Tomar Kwangjun Ahn Edward S. Hu Tim Pearce
 Pratyusha Sharma Akshay Krishnamurthy Riashat Islam Alex Lamb
 John Langford

Microsoft Research

Abstract

Transformers replace recurrence with a memory that grows with sequence length and self-attention that enables ad-hoc lookups over past tokens. Consequently, they lack an inherent incentive to compress history into compact latent states with consistent transition rules. This often leads to learning solutions that generalize poorly. We introduce **Next-Latent Prediction (NextLat)**, which extends standard next-token training with *self-supervised* predictions in the latent space. Specifically, NextLat trains a transformer to learn latent representations that are predictive of the next latent state given the next token. Theoretically, we show that these latents provably converge towards *belief states*, compressed information about the history necessary to predict the future. This simple auxiliary objective injects a recurrent inductive bias into transformers while leaving their architecture, parallel training efficiency, and inference unchanged. NextLat effectively encourages transformers to form compact internal world models with coherent belief states and transition dynamics—crucial properties not guaranteed by standard next-token prediction alone. Empirically, across benchmarks in world modeling, reasoning, planning, and language modeling, NextLat demonstrates significant gains over standard next-token prediction and other baselines in downstream accuracy, representation compression, and lookahead planning. Furthermore, NextLat enables *variable-length self-speculative decoding*, accelerating inference by up to $3.3\times$ in language modeling. NextLat offers a simple yet effective paradigm for learning compact, predictive representations in transformers that generalize better. Our code is available at <https://github.com/JaydenTeoh/NextLat>.

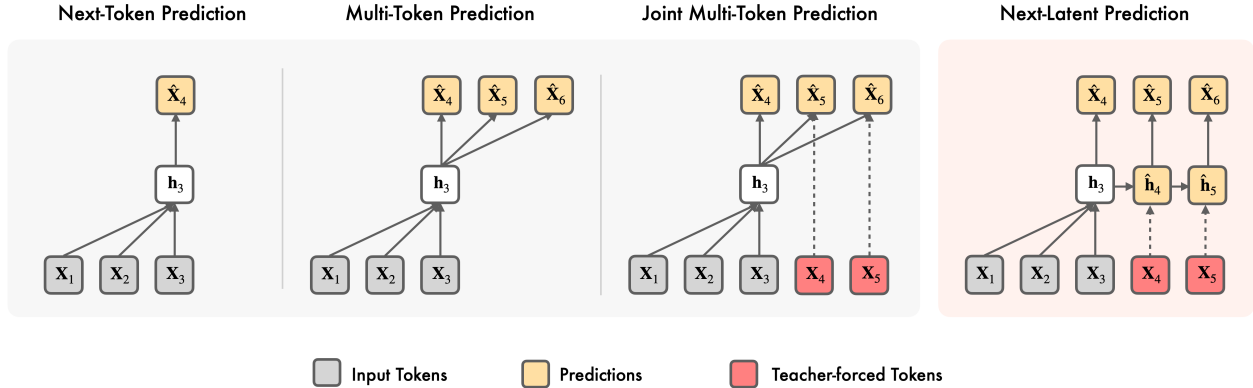


Figure 1: **Illustration of different predictive mechanisms.** Other methods supervise only in token space, leaving intermediate latent representations implicit. In contrast, NextLat explicitly trains the model to predict the next latent state (h_{t+1}) given the current latent state (h_t) and next token (X_{t+1}). Token supervision is then applied to the latent prediction ($\hat{h}_{t+1}$). Therefore, accurate multi-token predictions emerge from faithful latent dynamics modeling.

*Correspondence to: jayden_t[at]mit[dot]edu

1 Introduction

Ptolemy’s geocentric model was able to accurately predict observations of the solar system from Earth’s viewpoint, yet it was structurally convoluted: at times implying that the Moon came twice as close to Earth as other times. It was later supplanted by Copernicus’s simpler, more compact heliocentric model, which provided accurate predictions that generalized beyond Earth’s perspective. In learning theory, it is well known that simpler explanations of training observations tend to generalize better [Blumer et al., 1986, Langford and Schapire, 2005].

Modern transformers [Vaswani et al., 2017] stand in contrast to this principle. By replacing recurrence with a memory that scales with sequence length and self-attention that enables flexible lookups over past tokens, they achieve exceptional parallelization and predictive power. Yet, this very capability removes any inherent pressure to compress history into compact latent representations with consistent update rules. As a result, transformers often learn complex, task-specific shortcuts that fit the training data well but generalize poorly [Anil et al., 2022, Dziri et al., 2023, Liu et al., 2023, Wu et al., 2024]. How can we encourage transformers to form simpler, more principled explanations that avoid such shortcuts? A natural approach is to reinstate a key property of recurrent models: the ability to learn *compact world models* that channel future prediction through compressed representations of the past. We will show that this inductive bias can be reintroduced while also retaining the parallel training efficiency of transformers.

In this paper, we introduce **Next-Latent Prediction (NextLat)**, which extends the standard next-token prediction objective with self-supervised predictions in latent space. NextLat jointly trains a transformer and a latent dynamics model: the transformer learns to encode past tokens into compact latent summaries such that the dynamics model can predict the transformer’s next latent state given only the current latent state and the next token (i.e., the “action”). This objective encourages the transformer to form a compact internal world model with coherent recurrent-like dynamics, while avoiding the sequential processing overhead of recurrent architectures. Importantly, NextLat leaves the transformer’s architecture, inference procedure, and parallel training efficiency unchanged, adding only a lightweight auxiliary loss on latent representations during training. By augmenting sparse one-hot token targets with dense latent-state supervision, NextLat provides richer learning signals and improves data efficiency compared to next-token prediction and other token-level supervision methods. Our approach is inspired by the *self-predictive learning* paradigm in reinforcement learning (RL), a family of algorithms that learn representations by minimizing the prediction error of their own future latent states [Tang et al., 2023, Ni et al., 2024].

Beyond representation learning, latent dynamics also provides an inference advantage: future states can be recursively predicted directly in latent space without invoking the main transformer. This recursive multi-step lookahead using the lightweight dynamics model enables *variable-length self-speculative decoding*: the model can speculate a flexible number of future tokens and accelerate inference without requiring separate multi-token prediction heads.

The core contributions of this paper are threefold. First, we establish a theoretical foundation showing that NextLat provably shapes transformer representations into *belief states*—compact summaries of past information sufficient for predicting future observations. Such representations are important for planning and generalization, yet are not guaranteed to emerge from next-token prediction alone. Second, we present a practical implementation of NextLat that preserves the transformer’s architecture, inference procedure, and parallel training efficiency. Finally, we empirically demonstrate NextLat’s effectiveness across diverse domains spanning world modeling, reasoning, planning, and language modeling. Our results show that NextLat improves representation compactness, lookahead planning, and downstream accuracy over standard next-token prediction and other baselines. In language modeling, NextLat achieves up to $3.3\times$ faster inference through variable-length self-speculative decoding. Together, these results position NextLat as an efficient framework for learning compact, predictive, and generalizable representations in transformers.

2 Related Work

We are motivated by a long line of prior works in representation learning for prediction and control. Of close interest are self-supervised learning methods, belief states for decision making, and world models.

Self-Supervised Learning. Self-supervised learning (SSL) is a framework for learning from unlabeled data, where a model generates its own supervisory signals from the structure of raw inputs. Across modalities such as vision, audio, and time series, SSL has proven highly effective for pretraining useful features, enabling downstream transfer that rivals, or even surpasses, models trained on labeled data [Liu et al., 2022, Balestrierio et al., 2023, Zhang et al., 2024]. There are several approaches to SSL. Our method falls under *self-predictive representation learning*, which jointly learns latent representations and a transition function that models how these representations evolve over a sequence. Self-prediction has driven state-of-the-art advances in RL [Gelada et al., 2019, Zhang et al., 2020, Ye et al., 2021, Schwarzer et al., 2021, Hansen et al., 2022]. However, latent-space SSL remains underexplored in language modeling. A recent effort, LLM-JEPA [Huang et al., 2025], minimizes distances between embeddings of paired text–code data, but relies on manually curated pairs and therefore does not generalize to raw text. In contrast, our method introduces a fully self-supervised latent prediction objective requiring no paired data, making it broadly applicable for training transformers across arbitrary sequence modeling domains and data sources.

Belief States. In both sequence modeling and RL, models must reason over long histories of observations. To mitigate this curse of dimensionality, prior work focuses on compressing history into latent representations that capture all information necessary for future prediction. In RL, this latent summary is formalized by Kaelbling et al. [1998] as a *belief state*, defined as: “a sufficient statistic for the past history . . . no additional data about its past actions or observations would supply any further information about the current state of the world”. In stochastic control, the same notion of sufficient statistics appears as “information state” [Striebel, 1965]. The idea of sufficient statistics is also key to learning state abstractions in RL [Li et al., 2006]. While recurrent neural networks naturally enforce such compression, transformers have no such constraint—their internal state, or memory, grows linearly with sequence length. Recently, Belief State Transformers (BST; [Hu et al., 2025]) extended the notion of belief states to transformers, and demonstrated benefits in planning tasks. Compared to BST, NextLat learns belief states without requiring a separate transformer and is much more computationally efficient. We compare these methods further in Section 5.

World Models. Loosely, a world model is an internal predictive model of how the world works, with varying interpretations across cognitive science [Craik, 1967, Johnson-Laird, 1983], neuroscience [Miall and Wolpert, 1996, Friston, 2010], control theory [Francis and Wonham, 1976, Conant and Ross Ashby, 1970] and reinforcement learning [Sutton, 1991, Schmidhuber, 1990, Ha and Schmidhuber, 2018]. Whether transformer language models implicitly learn world models remains a debate; some studies report emergent world understanding [Patel and Pavlick, 2022, Li et al., 2023, Gurnee and Tegmark, 2024], while others find incoherent world structure [Vafa et al., 2024, 2025]. However, successful world modeling approaches such as *MuZero* [Schrittwieser et al., 2020] for achieving superhuman performance in video and board games, *Dreamer* [Hafner et al., 2019, 2021, 2025] for model-based RL, and *Genie* [Bruce et al., 2024] for interactive video generation share a common principle: they learn a latent dynamics model that takes a latent state (an encoding of past observations), an action, and predicts the next latent state. Yet, learning such latent dynamics for transformer-based language modeling remains underexplored. NextLat addresses this gap by explicitly learning a latent dynamics model that governs how the transformer’s latent states evolve given new tokens (i.e., “actions”), enabling the transformer to learn compact latent abstractions of the world with consistent dynamics.

Beyond Next-Token Prediction. In the domain of language modeling, a growing body of work has highlighted the myopic nature of the next-token prediction objective, which limits the model’s capability in downstream tasks such as planning and reasoning [Bachmann and Nagarajan, 2024, Nagarajan et al., 2025]. Recent works have also found improvements from richer supervision signals that predict further into the future [Gloeckle et al., 2024, Liu et al., 2024, Hu et al., 2025, Ahn et al., 2025]. However, these approaches operate predominantly in the token space. NextLat takes a different approach: it shifts prediction into the latent space, enforcing coherent dynamics over the model’s latent representations rather than its token outputs. As we discuss later in Section 3.1, this latent-space supervision provides richer gradient signals than token-level supervision.

Speculative Decoding. Speculative decoding [Leviathan et al., 2022, Chen et al., 2023] accelerates inference by using a lightweight draft model to propose multiple tokens, which are then verified in parallel by a target model. Closely related, Li et al. [2024a,b] train a draft model on top of a frozen language model to predict its high-level latent features.

Unlike our approach, this is done post-hoc, whereas we study latent prediction as a pretraining objective to directly shape representations. Moreover, their draft model relies on full attention over past features and therefore does not learn belief states, i.e., compact summaries of past tokens. *Self-speculative decoding* removes the need for a separate draft model by having a single model act as both draft and verifier. A common implementation uses the multi-token prediction (MTP) objective, where the model is trained to predict multiple future tokens that can be verified in parallel [Gloeckle et al., 2024]. However, MTP operates in token space and is therefore *typically* constrained to a fixed speculative horizon determined during training. In contrast, NextLat learns a latent dynamics model that can be recursively composed in latent space, enabling *variable-length self-speculative decoding* and faster inference.

3 Methodology: Next-Latent Prediction

In this section, we introduce a simple, yet powerful, method for learning belief states in transformers via next-latent prediction (or more specifically, via *next-hidden state prediction*¹). We begin by defining belief states in sequence modeling.

Definition 3.1 (Belief states in sequence modeling). Let $X_{1:T}$ denote a token sequence $X_1, \dots, X_T$. A random variable $\mathbf{b}_t = g(X_{1:t})$ is a *belief state* for the history $X_{1:t}$ if, for every bounded measurable function f of the future,

$$\mathbb{E}[f(X_{t+1:T}) \mid \mathbf{b}_t] = \mathbb{E}[f(X_{t+1:T}) \mid X_{1:t}] \quad \text{a.s.}$$

Equivalently, $\mathbf{b}_t$ is a *sufficient statistic* [Striebel, 1965] of the history $X_{1:t}$ for predicting the future tokens, i.e., from which we can sample from the distribution $\mathbb{P}(X_{t+1:T} \mid X_{1:t})$. Next, we describe how next-latent prediction enables transformers to learn belief states and improves data efficiency.

3.1 Why Next-Latent Prediction?

Here we analyze an idealized next-latent prediction transformer which successfully optimizes both next-token prediction and next-latent prediction with respect to an underlying data distribution.

Theorem 3.2. *Consider the joint learning of three components:*

1. a transformer with parameters θ that produces hidden states $\mathbf{h}_t$ at each time step t ,
2. an output head p_θ modeling the next-token distribution, and
3. a latent dynamics model p_ψ modeling the transition dynamics of the transformer’s hidden states.

If NextLat successfully optimizes the following objectives:

$$\text{(Next-Token Consistency):} \quad p_\theta(X_{t+1} \mid \mathbf{h}_t) = \mathbb{P}(X_{t+1} \mid X_{1:t}), \quad (1)$$

$$\text{(Transition Consistency):} \quad p_\psi(\mathbf{h}_{t+1} \mid \mathbf{h}_t, X_{t+1}) = \mathbb{P}(\mathbf{h}_{t+1} \mid X_{1:t+1}), \quad (2)$$

then $\mathbf{h}_t$ must be a belief state for the sequence $X_{1:t}$. Note that the right-hand side of Equation 2 is the transition law induced by the transformer’s weights².

Proof Sketch. A formal proof by backward induction is provided in Section B. Intuitively, optimizing for next-token (Equation 1) and transition (Equation 2) consistency ensures existence of measurable maps, i.e., p_θ and p_ψ , that allow recursive decoding of future tokens from $\mathbf{h}_t$:

$$\mathbf{h}_t \xrightarrow[\text{decode token}]{p_\theta} X_{t+1} \xrightarrow[\text{update state}]{p_\psi} \mathbf{h}_{t+1} \xrightarrow[\text{decode token}]{p_\theta} X_{t+2} \xrightarrow[\text{update state}]{p_\psi} \mathbf{h}_{t+2} \cdots \xrightarrow{p_\theta} X_T.$$

¹In the sequence modeling literature, intermediate latent representations are often referred to as “hidden states”. To disambiguate, we use the term “latent state” to broadly refer to learned representations within the transformer’s residual stream, and “hidden state” to refer to a subset of this representation—specifically, the final layer’s output at each time step (i.e., the pre-logit activations).

²We adopt a probabilistic formulation to retain generality with respect to stochastic transformer models, e.g. Fleuret [2025].

For these maps to exist, and be learned, $\mathbf{h}_t$ must jointly optimize toward a belief state—a sufficient statistic for the history to predict the future.

Remark. Optimizing only next-token consistency (i.e., Equation 1) in standard autoregressive transformers does not guarantee that $\mathbf{h}_t$ forms a belief state (see Theorem 3 in Hu et al. [2025]). Intuitively, self-attention enables ad-hoc lookup of past tokens, so there is no pressure to compress all necessary information about the past into compact latent summaries at every time step.

Better Data Efficiency. A distinctive feature of NextLat is the richness of its learning signal. In prior methods that supervise in token space (see Figure 1), the learning signal is only anchored to the next token, or multiple tokens. NextLat additionally supervises in latent space. Specifically, the model is trained to predict its own next hidden state $\mathbf{h}_{t+1}$, which parameterizes the full predictive distribution over X_{t+2} . This shifts supervision from individual one-hot token labels to distribution-level alignment. Moreover, because the latent dynamics compose recursively—each latent is trained to predict the next— $\mathbf{h}_{t+1}$ implicitly carries information about future states $\mathbf{h}_{t+2}, \mathbf{h}_{t+3}, \dots$. As a result, NextLat not only provides learning signals that are dense in the vocabulary space, but also propagates information about future tokens into earlier representations. By augmenting sparse one-hot targets with dense latent-state supervision, NextLat extracts more learning signal from each training sequence, leading to improved data efficiency.

3.2 Learning to Predict Next-Latent States

We now describe the practical implementation of NextLat, which augments standard next-token prediction with a self-supervised predictions in the latent space. Our NextLat implementation operates primarily on the hidden states (i.e., the final-layer outputs) as they provide compact, fixed-dimensional vectors through which gradients can be propagated through the entire transformer efficiently. As usual, we optimize the transformer and output head for next-token prediction (Equation 1) using the cross-entropy loss:

$$\mathcal{L}_{\text{next-token}}(\theta) = \mathbb{E}_{t < T} [-\log p_\theta(X_{t+1} | \mathbf{h}_t)].$$

NextLat additionally enforces transition consistency (Equation 2) of the hidden states by introducing a latent dynamics model p_ψ that predicts the next hidden state $\mathbf{h}_{t+1}$ directly from $(\mathbf{h}_t, X_{t+1})$. For a deterministic transformer model, $\mathbb{P}(\mathbf{h}_{t+1} | \mathbf{h}_t, X_{t+1})$ is a Dirac distribution, and we can optimize p_ψ via regression³. Moreover, observe that an ideal latent dynamics model should admit recursive consistency: its one-step map should compose correctly across multiple steps. Let $\hat{\mathbf{h}}_{t+d} = p_\psi(\mathbf{h}_t, X_{t+1:t+d})$ denote the recursive rollout of p_ψ over a d -step horizon using teacher-forced tokens $X_{t+1:t+d}$. We supervise all d intermediate rollouts using the Smooth L1 loss:

$$\mathcal{L}_{\text{next-h}}(\theta, \psi; d) = \mathbb{E}_t \left[\frac{1}{d} \sum_{i=1}^d \text{SmoothL1Loss}(\text{sg}[\mathbf{h}_{t+i}], \hat{\mathbf{h}}_{t+i}) \right], \quad (3)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator, used to prevent representational collapse in self-predictive learning [Ni et al., 2024]⁴. **Note that belief state convergence (i.e., Theorem 3.2) already holds for $d = 1$. Multi-step supervision serves only to provide richer learning signal.** To further align the semantics of predicted states $\hat{\mathbf{h}}$ with true states, we introduce a complementary KL objective enforcing agreement in token-prediction space:

$$\mathcal{L}_{\text{KL}}(\theta, \psi; d) = \mathbb{E}_t \left[\frac{1}{d} \sum_{i=1}^d D_{\text{KL}} \left(p_\theta^{\text{sg}}(\cdot | \text{sg}[\mathbf{h}_{t+i}]) \parallel p_\theta^{\text{sg}}(\cdot | \hat{\mathbf{h}}_{t+i}) \right) \right], \quad (4)$$

where the output head $p_\theta^{\text{sg}}(\cdot)$ is frozen so that gradients flow only through the latent dynamics model. This KL acts similarly to *knowledge distillation* [Hinton et al., 2015], providing soft supervision that guides learning of p_ψ . It also resembles *observation reconstruction* in self-predictive RL [Subramanian et al., 2022, Ni et al., 2024], encouraging $\hat{\mathbf{h}}_{t+i}$ to reproduce the distribution over next observations (i.e., the output head’s logits).

³If considering a stochastic transformer model, p_ψ can be optimized through variational inference.

⁴Technically speaking, the next-token prediction objective already provides grounding against representational collapse. However, in our ablations, we empirically observe better performance when applying the stop-gradient.

Overall Objective. The final NextLat objective combines all components, minimizing the following loss:

$$\mathcal{L}_{\text{NextLat}}(\theta, \psi; d, \lambda_{\text{next-h}}, \lambda_{\text{KL}}) = \mathcal{L}_{\text{next-token}}(\theta) + \lambda_{\text{next-h}} \mathcal{L}_{\text{next-h}}(\theta, \psi; d) + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}}(\theta, \psi; d), \quad (5)$$

where $\lambda_{\text{next-h}}, \lambda_{\text{KL}} > 0$ are scalar coefficients. Importantly, during inference, the learned transformer can decode independently; p_ψ is only needed during training to shape the transformer representations.

In the experiments that follow, we parameterize p_ψ using simple MLPs as our goal is to demonstrate that NextLat yields significant performance gains over baselines even without sophisticated latent dynamics architectures. Additional implementation details and ablations of key NextLat design choices are provided in [Sections C](#) and [D](#), respectively. To illustrate the simplicity of our approach, we also include a PyTorch-style pseudocode of the NextLat objective in [Algorithm 1](#).

3.3 Variable-Length Self-Speculative Decoding

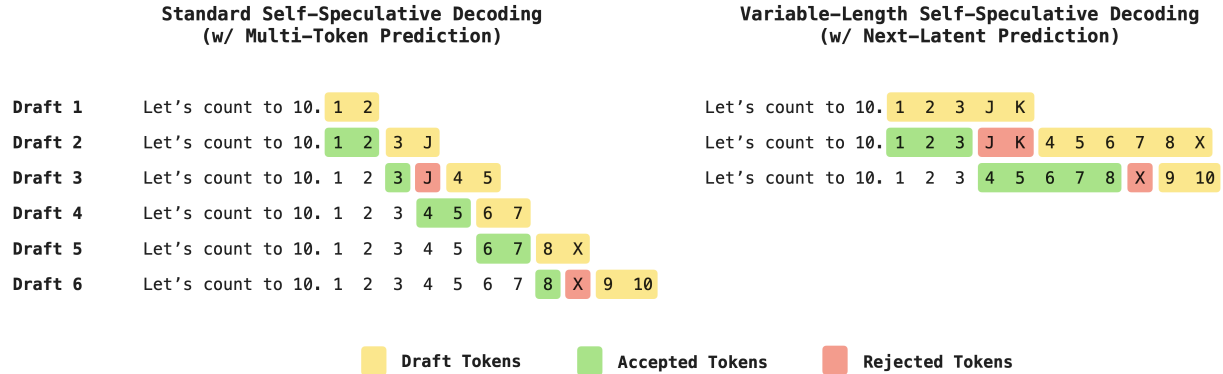


Figure 2: Illustration comparing self-speculative decoding with multi-token prediction (MTP) vs. NextLat. MTP uses a fixed draft length per step (e.g., $d = 2$ tokens here), whereas NextLat enables *variable-length self-speculative decoding*, reducing draft-verification cycles and accelerating inference.

In [Figure 2](#), we illustrate the difference in self-speculative decoding capabilities between MTP methods and NextLat⁵. MTP models are trained to predict the next d tokens and are therefore limited to drafting at most d tokens per draft-verification cycle. In contrast, even when trained with just $d = 1$ (i.e., no multi-step supervision), NextLat can recursively compose predictions via its latent dynamics:

$$\mathbf{h}_t \xrightarrow[\text{decode token}]{p_\theta} X_{t+1} \xrightarrow[\text{update state}]{p_\psi} \mathbf{h}_{t+1} \xrightarrow[\text{decode token}]{p_\theta} X_{t+2} \xrightarrow[\text{update state}]{p_\psi} \mathbf{h}_{t+2} \cdots,$$

enabling variable-length drafting during self-speculative decoding. We demonstrate in [Section 4.4](#) that, even with shallow training horizons ($d = 1, 2$) in the language domain, NextLat’s latent dynamics remain coherent far beyond the training horizon, enabling longer drafts and much faster inference than MTP baselines.

4 Experiments

Modeling coherent latent dynamics and compact beliefs about the underlying data-generating process is fundamental to both algorithmic and human reasoning. Therefore, in this section, we evaluate NextLat on four key axes where such capabilities matter most: world modeling, reasoning, planning, and language modeling.

⁵The figure illustrates the potential for adaptive draft lengths with NextLat. In the scope of this work, however, we do not implement adaptive drafting strategies; once selected, the draft length remains static throughout decoding. Instead, we exploit the ability to *vary* these fixed draft lengths to extend beyond the training horizon d in order to maximize inference speedup.

Our baseline comparisons include transformer-based belief-learning methods, i.e., BST [Hu et al., 2025] and JTP [Ahn et al., 2025]. Further discussions and detailed comparisons with these methods are provided in Section 5 and Appendix A. For completeness, we also report the performances of standard next-token prediction (GPT) and multi-token prediction (MTP). The MTP baseline follows the implementation of Gloeckle et al. [2024], and we follow the `nanoGPT` codebase [Karpathy, 2022] for our decoder-only transformer implementations. Hereafter, we use the term “horizon” to refer to the multi-step prediction horizon d in JTP, MTP and NextLat, and we match horizon across these methods in all experiments to ensure fair comparisons. For specific experiment details such as hyperparameters, evaluation procedure, etc., please refer to Section F.

4.1 World Modeling

	Next-Token Test (↑)	Valid Trajectories (↑)	Sequence Compression (↑)	Effective Latent Rank (↓)	Detour Robustness (↑)
GPT	100%	97.0%	0.65	160.1	85.0%
MTP	100%	98.1%	0.64	57.7	95.0%
JTP	100%	97.1%	0.32	215.8	87.0%
NextLat	100%	98.7%	0.71	52.7	95.0%
True world model	100%	100%	1.00	—	100%

Table 1: Comparison of GPT, MTP, JTP, and NextLat trained on Manhattan taxi rides against the true world model across several metrics.

Vafa et al. [2024] introduced a dataset of turn-by-turn taxi rides in Manhattan, where the true world model (i.e., the city’s street map) is visually interpretable. Their study revealed that transformers trained on such trajectories can achieve near-perfect next-token accuracy, yet their internal maps remain incoherent; they reconstruct streets with impossible orientations and even flyovers above other roads.

Setup. We use the *random walks* dataset from Vafa et al. [2024], which consists of random Manhattan traversals (91M sequences, 4.7B tokens) between taxi pickup and dropoff points. Models are trained for 6 epochs (vs. 1 epoch in their study) as we observe performance does not converge within a single epoch. Due to its high computational cost, BST is excluded from this benchmark. For JTP, MTP, and NextLat, we set the multi-step prediction horizon at $d = 8$. We evaluate world-modeling performance using five comprehensive metrics:

1. Next-Token Test: Percentage of top-1 token predictions corresponding to legal turns under teacher-forcing on in-distribution validation sequences.
2. Valid Trajectories: Percentage of valid traversals for out-of-distribution (OOD) pickup–dropoff pairs.
3. Sequence Compression: Percentage of cases where the model produces identical continuations when prompted with two different traversals arriving at the same state and sharing the same destination.
4. Effective Latent Rank: Effective rank/dimension of hidden states measured as the exponentiated Shannon entropy of the normalized singular values [Roy and Vetterli, 2007]; lower values indicate better compression.
5. Detour Robustness: Percentage of valid traversals for OOD pickup–dropoff pairs when we substitute the model’s top-1 prediction with random detours (legal turns) 75% of the time.

The results are shown in Table 1. More details on training and evaluation are provided in Section F. We also refer motivated readers to Vafa et al. [2024] for further explanations of the next-token test, valid trajectories, sequence compression, and detour robustness metrics.

Results. Similar to the original study, all models achieved 100% accuracy on the next-token test. However, next-token accuracy is a limited diagnostic and cannot meaningfully assess the quality of a model’s learned world model. In Figure 3, we visualize each model’s internal map using the reconstruction algorithm proposed by Vafa et al. [2024]. Visibly, the transformer trained with NextLat exhibits an internal map more consistent with the true world model.

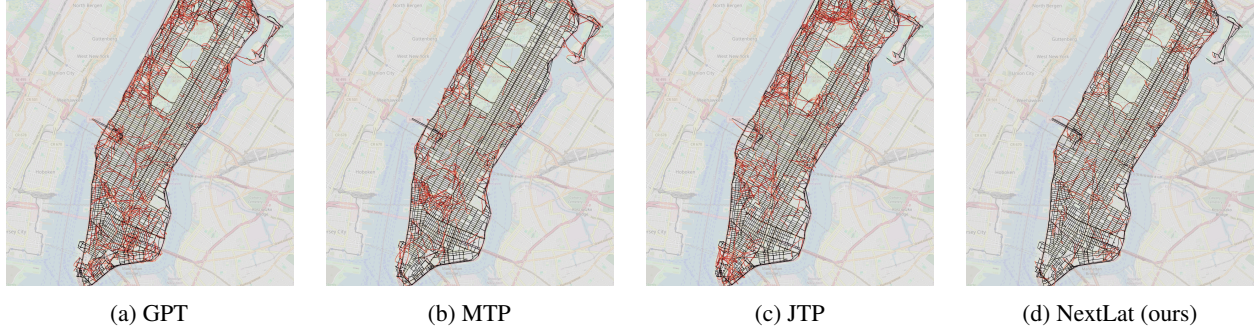


Figure 3: Reconstructed maps from transformers trained on Manhattan taxi rides using different objectives.

Although not perfect, its inconsistencies (red edges) are sparse and mostly local. Beyond this qualitative evidence, NextLat consistently outperforms all baselines across all metrics. On the trajectory validity and detour robustness metrics, NextLat demonstrates the **strongest generalization to OOD pickup-dropoff pairs**, even when random detours are introduced.

Next, we analyze the compactness of the learned world models using two compression metrics. A model that accurately captures the underlying states and transitions should assign identical continuations to trajectories that end in the same state (i.e., intersection in Manhattan). By this criterion, NextLat achieves the **highest sequence compression of 0.71**. The true Manhattan graph comprises only 4,580 intersections and 9,846 edges, and therefore an effective world model should require only a modest latent dimensionality. Indeed, NextLat has the **lowest effective latent rank of 52.7—over 3x smaller than GPT’s**. The combination of stronger planning performance and more compact latent representations reinforces the view that NextLat, by promoting belief state representations and coherent latent dynamics, enables transformers to learn substantially better world models—ones that are both accurate in their predictive structure and efficient in their internal representation of the environment.

4.2 Reasoning

Countdown [Countdown, 2025] is a mathematical reasoning task and a generalized version of the Game of 24, which even frontier models such as GPT-4 [Achiam et al., 2023] have struggled with, achieving 4% by default [Yao et al., 2023]. The goal of the task is to combine a set of given numbers with basic arithmetic operations ($+$, $-$, $\times$, $\div$) to obtain a target number. For example, given the numbers $\{90, 8, 20, 50\}$, the target number 24 can be obtained using the following sequence of equations: $90 \times 8 = 720$, $50 - 20 = 30$, $720 \div 30 = 24$. Countdown poses a difficult combinatorial search problem due to its large branching factor and the need to efficiently explore the solution space to reach the target number.

Setup. Following Gandhi et al. [2024], we generate 500k training problems with target numbers ranging from 10 to 100 and reserve 10% of the targets for out-of-distribution evaluation. During both training and testing, we insert eight ‘pause tokens’ [Goyal et al., 2023] after the target number, allowing models additional computation to plan before generating a solution. Performance is measured as the percentage of 10k test problems for which a model produces a valid sequence of equations that correctly reaches the target number. The reported results average over three random seeds per baseline.

Model	Horizon (d)	Accuracy (%)
GPT	–	33.1
BST	–	42.3
MTP	1	39.2
	4	49.7
	8	57.3
JTP	1	39.0
	4	49.4
	8	55.0
NextLat	1	54.8
	4	<u>57.6</u>
	8	58.7

Figure 4: Performance on Countdown. Best result is **bolded**, and second best is underlined.

Results. As shown in Figure 4, NextLat consistently outperforms all baselines in Countdown. Notably, even with a shallow supervision horizon of $d = 1$, NextLat substantially surpasses MTP and JTP trained with the same horizon (**>35.7% improvement**). To better understand this gap, we analyzed the equations generated by each model and evaluated their equation validity, i.e., whether the computed left-hand side equals the right-hand side. As shown in Figure 5, most calculation errors occur in the final equation (Eq. 3). This indicates that the model’s lack of planning capability results in its realization of being unable to achieve the goal only at the end. Unable to revise earlier missteps, it forces an invalid final equation to match the desired outcome—a behavior termed *the regretful compromise* by Ye et al. [2025]. NextLat demonstrates **stronger lookahead planning**: even with $d = 1$, it achieves substantially higher mean validity in the final equation (54.8%) compared to the next best baseline (42.3%). This suggests that the latent-state prediction objective may help the model anticipate long-range dependencies and form globally consistent plans, reducing the tendency to make myopic errors. NextLat also achieves leading performance with horizons $d = 4$ and 8.

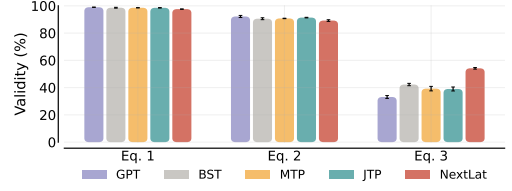


Figure 5: Validity of equations (i.e., LHS = RHS) generated on Countdown. All models in this plot use $d = 1$.

4.3 Planning

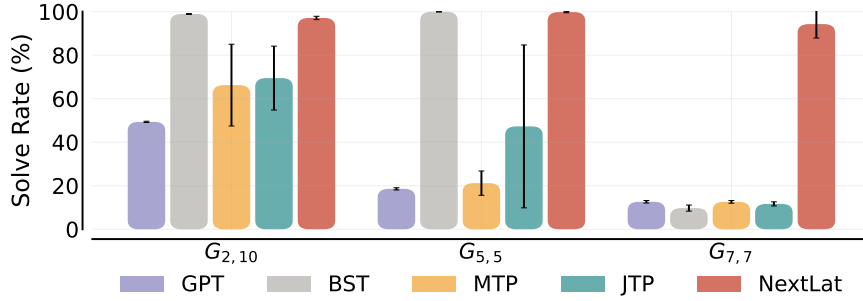


Figure 6: Accuracy on Path-Star graph task. Unlike the baselines, NextLat maintains close to 100% solve rate for all graphs.

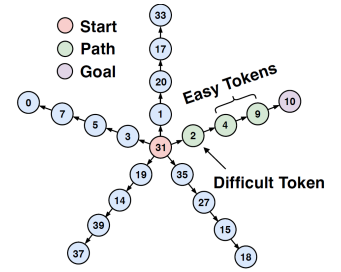


Figure 7: Illustration of a $G_{5,5}$ Path-Star graph [Bachmann and Nagarajan, 2024].

A Path-Star graph [Bachmann and Nagarajan, 2024] $G_{d,\ell}$ consists of a center node and d disjoint arms, each consisting of $\ell - 1$ nodes. Figure 7 depicts an instantiation of the $G_{5,5}$ topology. A training instance is a tokenized sequence that contains the edge list, the start and end nodes, and the correct path from start to end. This task represents a minimal instance of lookahead planning, a core capability underlying more complex behaviors such as storytelling. Yet, despite its apparent simplicity, next-token prediction models struggle to solve it.

Setup. Following Bachmann and Nagarajan [2024], we generate 200k training samples and set $N = 100$, such that node values in each graph are randomly drawn from $1, \dots, 100$. For MTP, JTP, and NextLat, we set the multi-step prediction horizon to $d = \ell - 2$, ensuring that the target (end) node lies within the multi-step prediction horizon of the center node. We evaluate performance across three graph configurations: $G_{2,10}$, $G_{5,5}$, and $G_{7,7}$ across five random seeds per baseline.

Results. As shown in Figure 6, NextLat maintains close to 100% solve rate for all topologies of the Path-Star graphs. BST, while able to solve $G_{2,10}$ and $G_{5,5}$, begins to fail at the larger graph $G_{7,7}$. Note that our results differ from that presented in Hu et al. [2025] (BST) and Ahn et al. [2025] (JTP). In their setup, they use a much smaller problem settings of $N = 50$, and generate a fresh batch of graphs every iteration. On the other hand, we use the original (and more difficult) setup which has a fixed sample size of 200k and $N = 100$.

The Path-Star graph task is specifically designed to reveal the myopic behavior of teacher-forced next-token prediction models, which tend to exploit local shortcuts instead of learning to perform lookahead planning necessary to solve the task. This phenomenon, termed the *Clever Hans cheat* [Bachmann and Nagarajan, 2024], is related to the difficulty of learning parity [Hu et al., 2025] and has motivated methods such as BST and JTP that attempt to mitigate shortcut learning through multi-token predictions. However, these approaches operate in token space, making them still susceptible to local n -gram regularities that do not capture the underlying transition structure required for long-horizon planning. In contrast, NextLat performs prediction in latent space, enforcing recurrent transition consistency at the representation level. NextLat’s success across all graph configurations, unlike MTP, JTP, and BST, suggests that latent-space prediction better avoids shortcut learning and yields more generalizable solutions. Furthermore, given the limited training samples (i.e., 200k) in this task, data efficiency is crucial. NextLat’s strong performance in this low-data regime suggests that it extracts more useful supervision from each training sequence than token-prediction baselines, consistent with our analysis in Section 3.1.

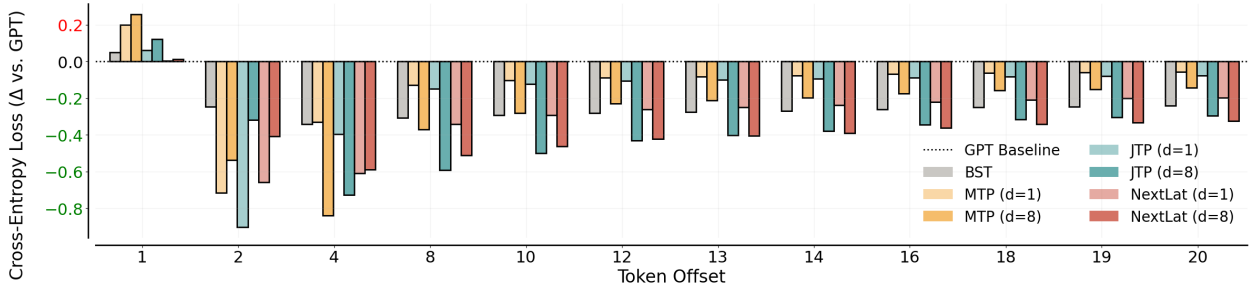


Figure 8: Cross-entropy loss difference relative to GPT, obtained from linear probes trained on frozen hidden states to predict tokens at varying token offsets (x-axis values) ahead. Lower values indicate better predictive performance.

Next, we compare the models on TinyStories [Eldan and Li, 2023], a dataset consisting of synthetic short stories. Storytelling is inherently a long-horizon planning problem; a coherent narrative requires maintaining persistent entities, tracking causal relationships, resolving conflicts and delayed resolutions, and satisfying narrative constraints across many timesteps. Generating such sequences therefore depends not only on next-token prediction, but on belief-state-like abstractions that encode information predictive of future story trajectories.

Setup. Following Hu et al. [2025], we tokenize the dataset of 2.7 million stories into a vocabulary of 1,000 tokens and construct training sequences of length 256. All models are trained for 100k steps, which is sufficient for convergence. We include comparisons of transformers trained using MTP, JTP, and NextLat with multi-step prediction horizons $d \in \{1, 8\}$. After training, we freeze the model parameters and train 20 independent linear probes, one per token offset, to predict tokens at offsets $1, \dots, 20$ steps ahead from the hidden states of the frozen models using the same dataset. This allows us to assess whether the models’ representations encode information predictive of future tokens, or just local token correlations.

Results. We plot the difference in probe performance relative to probes trained on GPT’s hidden states in Figure 8. For clarity, we display only selected token offsets here (see Figure 16 in the appendix for full results). Observe that the additional token-level prediction objectives in BST, MTP, and JTP consistently cause significant degradation in next-token prediction (i.e., token offset = 1). Moreover, probe performance on JTP and MTP representations declines sharply with increasing token offset. This indicates that these multi-token prediction models, lacking guarantees of learning belief states, could encode information useful only for short-horizon prediction. In contrast, NextLat matches GPT’s next-token performance across both $d \in \{1, 8\}$ and exhibits the strongest long-range predictive capability (up to 20 tokens ahead) for both $d = 1$ and $d = 8$. These results suggest that NextLat’s latent-state objective induces belief-like representations that encode predictive information about future events—an ability essential for maintaining coherence in long-range narrative generation tasks like TinyStories.

4.4 Language Modeling

Model	FW-Edu ppl ↓	Wiki. ppl ↓	LAMB. ppl ↓	LAMB. acc ↑	PIQA acc ↑	HellaS. acc ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc ↑	SIQA acc ↑	SciQ acc ↑	Avg.
GPT	10.52	17.93	20.26	42.07	73.45	58.79	<u>60.46</u>	68.18	39.16	42.32	86.10	58.82
JTP (d=1)	11.08	19.28	21.88	41.35	74.92	57.43	<u>58.64</u>	68.73	39.25	42.99	<u>87.30</u>	<u>58.83</u>
JTP (d=2)	11.18	19.60	22.11	41.37	73.34	56.84	59.98	68.86	38.57	43.35	86.70	58.63
MTP (d=1)	10.90	18.82	20.23	41.26	<u>74.32</u>	58.05	60.54	68.52	38.91	42.84	85.40	58.76
MTP (d=2)	11.00	18.61	<u>18.34</u>	<u>43.43</u>	72.80	57.92	59.35	68.35	39.08	41.97	86.60	58.69
NextLat (d=1)	<u>10.83</u>	<u>18.39</u>	19.77	41.08	73.07	<u>58.35</u>	59.27	<u>69.65</u>	<u>39.68</u>	<u>43.24</u>	86.00	58.79
NextLat (d=2)	10.88	18.44	17.83	43.86	73.61	57.79	59.20	69.74	40.10	41.91	87.50	59.21

Table 2: Downstream language modeling evaluation on 1.3B-parameter models trained on 100B FineWeb-Edu tokens. Best scores are in bold and second-best are underlined.

Setup. We pretrain 1.3B-parameter models on 100B tokens from the FineWeb-Edu dataset [Penedo et al., 2024], excluding BST due to its high computational cost. After pretraining, we use the LM Evaluation Harness [Gao et al., 2024] to evaluate the zero-shot accuracy of the models on multiple-choice language modeling benchmarks. We also evaluate the self-speculative decoding performance of the multi-step prediction models, i.e., JTP, MTP, and NextLat, across Wikipedia, Books, Code, and Math domains. For each dataset, we sample 1024 prompts of length 512 tokens and generate 512-token continuations using the speculative sampling algorithm of Leviathan et al. [2022]. For each model, we report the average number of accepted tokens per drafting step, as well as the inference speedup relative to standard autoregressive sampling from the base transformer, measured on 8×NVIDIA B200 GPUs. For multi-step prediction models, we primarily focus our analysis on training with horizons of $d = 1$ and $d = 2$. MTP and JTP operate in token space and are therefore usually limited to speculative decoding within the fixed multi-token horizon used during training. Our aim is to highlight the advantages of NextLat’s variable-length self-speculative decoding, which enables drafting beyond the training horizon. To this end, we vary NextLat’s speculative draft length between 2 and 10 tokens and report the highest inference speedup achieved for each domain.

Model	Wikipedia		Books		Code		Math	
	Speedup	Accepted Tokens	Speedup	Accepted Tokens	Speedup	Accepted Tokens	Speedup	Accepted Tokens
JTP (d=1)	1.46	0.96	1.47	0.97	1.47	0.98	1.46	0.97
JTP (d=2)	1.88	1.84	1.90	1.89	1.88	1.85	1.89	1.86
MTP (d=1)	1.38	0.91	1.39	0.95	1.40	0.97	1.39	0.95
MTP (d=2)	1.68	1.72	1.72	1.83	1.75	1.91	1.72	1.84
NextLat (d=1)	<u>2.68</u>	<u>3.52</u>	<u>2.72</u>	<u>3.64</u>	<u>2.29</u>	<u>2.66</u>	<u>2.30</u>	<u>2.72</u>
NextLat (d=2)	3.21	4.59	3.32	4.86	2.38	2.83	2.87	3.94

Table 3: Relative speedup and average accepted tokens per drafting steps over diverse domains. Note that “Accepted Tokens” excludes the next-token prediction which is always accepted.

Results. In Table 2, we present the language modeling perplexity (ppl) and zero-shot accuracy (acc) of the pretrained models across several benchmarks. Consistent with observations in Gloeckle et al. [2024], we do not observe significant improvements in multiple-choice task accuracy over standard next-token training (GPT) when using multi-token prediction objectives. NextLat ($d = 2$) does show a modest gain in average accuracy over GPT (59.21 vs. 58.82), but these improvements are not consistent across tasks. Larger model sizes might be necessary to see more significant improvements. Notably, NextLat **better preserves next-token perplexity compared to MTP and JTP** across FineWebEdu, Wikitext, and LAMBADA. This is consistent with our earlier observations on TinyStories in Section 4.3. Preserving high fidelity in next-token prediction is arguably important, as prior work shows that lower pretraining perplexity correlates with improved aggregate downstream and post-fine-tuning performance at larger model scales [Gadre et al., 2024, Zhang et al., 2026].

Next, in Table 3, we show the self-speculative decoding results. Observe that the average accepted tokens per drafting steps for NextLat far exceeds its training horizon d , indicating that the learned latent dynamics remains coherent over extended rollouts. This further highlights the strong long-range predictive capability of the induced belief state representations. Crucially, this ability to support variable-length self-speculative decoding enables NextLat to achieve **substantially higher inference speedups than MTP and JTP across all domains**. Figure 9 further shows speedup and cumulative acceptance rate versus draft length on the FineWeb-Edu validation set. Speedup increases sublinearly with draft length, reaching up to $3.3\times$, and fully valid (i.e., all tokens accepted) drafts persist even at length 10. This demonstrates a clear benefit to drafting tokens beyond the training horizon with NextLat. In Section F.6.1, we extend our comparisons to include JTP and MTP trained with larger horizons ($d = 4$). Note that the training cost of JTP and MTP increases substantially with larger d (see Table 4), making JTP ($d = 4$) and MTP ($d = 4$) significantly more expensive to train than NextLat ($d = 1, 2$). Still, even with longer multi-token supervision, these baselines still fail to surpass NextLat in speculative decoding performance. This highlights a key advantage of NextLat’s variable-length speculative decoding: it enables long speculative drafts while requiring training only at shallow, computationally efficient horizons.

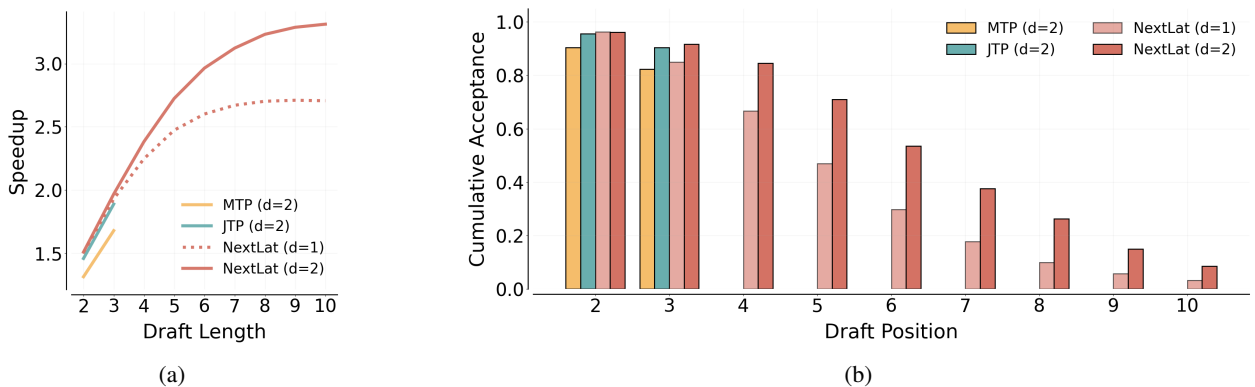


Figure 9: (a) Inference speedup ratio of JTP, MTP, and NextLat on the FineWeb-Edu validation set. (b) Cumulative acceptance rate of draft tokens (product of per-position acceptance probabilities), indicating how likely the entire speculative draft remains valid (i.e., all tokens accepted) as draft length increases.

5 Discussions

5.1 Comparison Against Current Approaches

	GPT	BST	MTP	JTP	NextLat
Training parameters ($d = 1, 2, 8$)	1.32B	2.57B	1.37B / 1.42B / 1.72B	1.34B	1.40B
Inference parameters	1.32B	1.32B / 2.57B	1.32B	1.34B	1.32B
Training steps/second ($d = 1, 2, 8$)	3.09	0.89	2.80 / 2.58 / 1.70	3.15 / 2.92 / 2.02	3.09 / 2.79 / 1.73
Gradients	$O(T)$	$O(T^2)$	$O(Td)$	$O(Td)$	$O(Td)$

Table 4: Comparison of training speed, parameter count and gradient signals provided on FineWeb-Edu pretraining. For training parameter numbers and training iterations/second, we report values for prediction horizons $d = 1, 2, 8$. If only a single value is shown, it means that the values are the same for all horizons. All training speeds are measured on a single NVIDIA B200 GPU with batch size of 33k tokens.

In this section, we discuss the advantages of NextLat compared to prior transformer learning approaches, focusing discussions on belief-learning methods, i.e., BST and JTP. We provide brief descriptions of these methods and their training objectives in [Section A](#). Motivated readers interested in further details beyond the scope of this work may refer to [Hu et al. \[2025\]](#) and [Ahn et al. \[2025\]](#). [Table 4](#) summarizes the training and inference parameters, training speed (in iterations per second), and gradient signal characteristics for each method on FineWeb-Edu pretraining, providing context for the discussions that follow.

Computational Costs. While BST benefits from $O(T^2)$ gradient signals per token sequence, this is a double-edged sword. Even with the optimized implementation of [Hu et al. \[2025\]](#), training remains extremely costly because gradients must be accumulated over all $O(T^2)$ predictions of different prefix–suffix pairs. Moreover, BST trains two transformer encoders, further increasing compute and parameter cost. During inference, BST also uses both transformer encoders, one for generation and the other for scoring the likelihood of the generated sequence. On FineWeb-Edu pretraining, BST is over $3\times$ slower than NextLat in training speed (see [Table 4](#)). Note that this result already uses 10% subsampling of prefix-suffix pairs to prevent out-of-memory issues; the training speed of BST would be much slower if all pairs were used for training. In contrast, NextLat with $d = 1$ incurs negligible overhead relative to GPT while achieving the same belief-state learning guarantees as BST.

We next compare the compute costs of the multi-step prediction methods, i.e., MTP, JTP, and NextLat. The MTP implementation of [Gloeckle et al. \[2024\]](#), as well as other variants such as the one introduced in [Liu et al. \[2024\]](#), require additional transformer layers as the prediction horizon d increases, whereas JTP and NextLat keep parameter counts fixed across horizons. JTP and NextLat exhibit similar training speeds for $d = 1$ ⁶, while MTP lags behind. At $d = 8$, JTP is substantially faster than NextLat because NextLat sequentially unrolls its latent dynamics model p_ψ to compute multi-step losses, while JTP computes them in parallel. Nonetheless, this modest increase in computation for NextLat yields substantially better performance than JTP across all benchmarks. Importantly, NextLat’s sequential computation remains far more efficient than that of recurrent neural networks (RNN), which we discuss further in [Section 5.2](#).

Belief State Learning. GPT and MTP lack any theoretical learning pressure to form belief-state representations, which means that they do not necessarily learn sufficient representations predictive of future observations. JTP can learn belief states but only under the restrictive condition that the prediction horizon satisfies $d \geq k$, where k denotes the observability horizon of the underlying data-generating process (see [Theorem A.1](#) in appendix). In long-context sequence modeling, however, the underlying process is often k -observable for a very large k , rendering this condition impractical. NextLat, on the other hand, learns belief-state representations independently of d and larger multi-step prediction horizons are used only to provide richer gradient signals. It also avoids the expensive $O(T^2)$ gradient computation required by BST to learn belief states. As such, NextLat stands as a simple, computationally efficient, and theoretically grounded alternative to existing belief-state learning approaches (i.e., BST and JTP) in autoregressive sequence modeling.

Myopic Nature of Token-level Predictions. Token-level supervision is often myopic. Next-token prediction transformers tend to prioritize short-term dependencies, and studies have shown that early training can often resemble n -gram modeling, which can delay or even prevent learning of the true Markov kernel [[Edelman et al., 2024](#), [Makkuva et al., 2025](#)]. [Bachmann and Nagarajan \[2024\]](#) further showed that the myopic nature of next-token prediction training can trap models in suboptimal local minima, undermining long-horizon planning. In our experiments, we find that adding additional token-level prediction objectives, as in BST, JTP, and MTP, not only degrades next-token performance but also fails to yield consistent gains in generalization. In contrast, NextLat, which emphasizes latent transition modeling as its primary objective, minimizes degradation in next-token performance and improves downstream generalization by encouraging the learning of predictive representations rather than shallow token-level correlations. Moreover, as discussed in [Section 3.1](#), latent-space supervision provides denser gradient signals than token-level supervision. Overall, NextLat offers a compelling alternative to conventional token-level prediction objectives for transformer training.

⁶JTP exhibiting a slightly faster training speed than GPT at $d = 1$ is likely an artifact of `torch.compile()` due to differences in the computation graph.

5.2 NextLat vs. Recurrent Neural Networks

Algorithmic reasoning requires capabilities most naturally understood through recurrent models of computation, like the Turing machine. However, strict recurrence imposes a sequential computation bottleneck at training time. NextLat introduces a *recurrent inductive bias* for the learned representations via latent transition prediction without turning the transformer into a strictly sequential model.

Training RNNs without recurrence. Intuitively, NextLat ($d = 1$) can be viewed as *fully parallel* co-training of a transformer and a recurrent neural network (RNN). The transformer forecasts latent states across the sequence in parallel, while the latent dynamics model (the RNN) is trained using only one-step latent predictions. This co-training process does not require any sequential computation across time.

To study this interaction, we consider the A_5 word problem, a difficult *state-tracking* task involving even permutations of five elements. The problem is NC^1 -complete and therefore inexpressible by constant depth, fixed-precision transformers, which are restricted to the $TC^0 \subset NC^1$ circuit complexity class [Merrill and Sabharwal, 2023]⁷. We train 2-layer transformers using next-token prediction (GPT) and next-latent prediction (NextLat, $d = 1$) on the A_5 word problem with a sequence length of 12 tokens (permutations). We then tested the transformers and the latent dynamics model (RNN) on their length generalization to sequences of length 36. Note that during inference, the RNN only uses the transformer backbone to initialize the hidden state for the first token, after which subsequent hidden state transitions and token predictions are generated independently by the RNN without invoking the transformer. Importantly, the RNN also has substantially fewer parameters than the transformer (2.62M vs. 6.43M parameters).

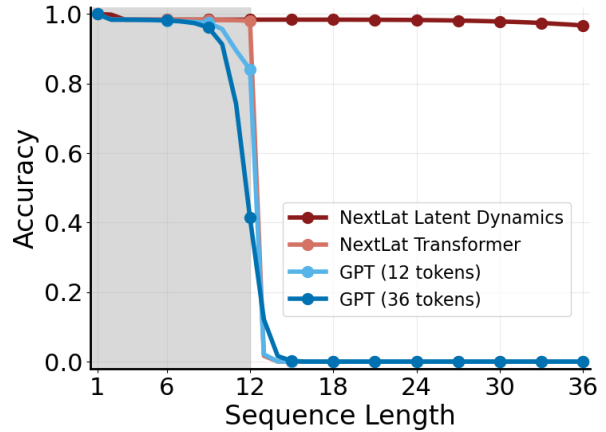


Figure 10: Length Generalization for A_5 word problem.

Figure 10 shows interesting results. First, the transformer trained with NextLat exhibits better state-tracking performance than GPT within the 12-token training horizon. More surprisingly, while the transformer itself fails to generalize beyond the 12-token horizon, the learned latent dynamics model successfully generalizes to 36-token sequences ($> 95\%$ accuracy). This is particularly striking because even a GPT model trained directly on 36-token sequences (“GPT (36 tokens)”) is unable to solve the task, consistent with previous results [Merrill et al., 2025]. In other words, although the transformer itself cannot solve the 36-token A_5 problem, it is nevertheless able to train an RNN that can.

From a circuit complexity perspective, solving problems in NC^1 requires transformers with $O(\log T)$ depth [Merrill and Sabharwal, 2023, Liu et al., 2023], where T is the sequence length. This explains why the shallow 2-layer transformers (both GPT and NextLat) are unable to solve problems beyond 12 tokens. What is surprising, however, is that under NextLat training, **the co-trained RNN can generalize far beyond the expressivity limits of the underlying transformer, despite being trained entirely in parallel**. This raises an intriguing question:

Given inputs of length T , can a transformer with $O(\log T)$ depth co-train (in parallel) an RNN that generalizes to sequences of length $\gg T$? More specifically, can the co-trained RNN learn NC^1 computations, even though it is supervised using representations from a transformer in TC^0 ?

Intuitively, this seems plausible. As long as the transformer has sufficient depth to express and forecast hidden states within the T -length training context window, it makes sense for the latent dynamics model (RNN) to learn reusable computational circuits that extrapolate far beyond it, as empirically suggested by Figure 10. In this view, NextLat may offer a mechanism for partially escaping the practical limitations imposed by the *parallelism tradeoff* [Merrill

⁷This assumes the widely held conjecture that $TC^0 \neq NC^1$.

and Sabharwal, 2023]: the transformer provides efficient parallel representation learning, while the RNN focuses on learning generalizable sequential update rules.

Computational Efficiency and Relation to Backpropagation Through Time. Now, we further compare NextLat against traditional RNNs trained using backpropagation through time. Traditional RNNs incur $O(T)$ sequential dependence during training. In contrast, NextLat adds only an additional sequential cost proportional to the rollout horizon $d \ll T$, corresponding to unrolling the latent dynamics model p_ψ for d steps. This allows NextLat to largely retain the transformer’s parallel training efficiency while also inheriting a recurrent inductive bias.

Conceptually, the one-step and multi-step prediction in NextLat resembles truncated backpropagation through time (TBPTT) in RNNs, with truncation windows of 1 and d , respectively. However, a key distinction lies in how gradients are propagated. In TBPTT for RNNs, gradient computation is truncated beyond the chosen window, yielding *biased* gradient estimates that lack convergence guarantees. In contrast, the **theoretical convergence of belief-state learning in NextLat, as shown in Theorem 3.2, is irrespective of d** , since it requires only one-step prediction optimality. Intuitively, NextLat performs full backpropagation through the transformer’s computation graph, where self-attention connects all past tokens. The latent dynamics model operates in an “outer loop”, aligning the transformer’s hidden states to be temporally consistent across steps. Crucially, this outer-loop supervision does not truncate gradient flow within the transformer, and therefore avoids bias with respect to d . Larger prediction horizons simply provide richer supervision and faster empirical convergence.

Expressivity of the Recurrence. Modern state-space models (SSMs), such as S4 and Mamba, implement efficient linear recurrent updates while remaining highly parallelizable [Gu et al., 2022, Gu and Dao, 2024]. In contrast to SSMs, the latent dynamics model p_ψ in NextLat can express nonlinear transitions. Moreover, NextLat does not explicitly perform recurrence in the forward computation. Instead, recurrent-like dynamics emerge implicitly through one-step or multi-step unrolling of the p_ψ and aligning successive hidden states via regression. This induces temporal consistency within the latent space without altering the transformer architecture. However, it is important to note that NextLat modifies the learned representations, not the underlying circuit complexity. The overall computational expressivity of the fixed-depth transformer trained with NextLat remains bounded by that of constant-depth threshold circuits, i.e., within the TC^0 complexity class [Merrill et al., 2022]. However, earlier results in Figure 10 question whether the co-trained latent dynamics model (RNN) is subject to the same TC^0 limitations.

Hybrids Attention Models. NextLat differs fundamentally from hybrid Transformer–SSM architectures [Park et al., 2024, Lieber et al., 2024, Ren et al., 2025], which explicitly interleave SSM and attention layers. NextLat requires no architectural changes and operates purely as an auxiliary training objective on the model’s latent representations. As such, it is broadly compatible with diverse sequence-modeling architectures. In fact, NextLat could complement hybrid models by encouraging compression and consistency in the residual attention pathways that fall outside the SSM’s recurrent structure, potentially improving representation efficiency and generalization.

6 Limitations and Future Work

While NextLat shows strong empirical performance, several limitations remain in our work. First, we do not explore the design space of the latent dynamics model; all experiments use simple MLPs to isolate and demonstrate the effectiveness of the core NextLat approach. More expressive architectures may further improve performance. We also do not study how the width of the hidden layers in the latent dynamics MLP affects learning, even though it effectively acts as a bottleneck that constrains belief-state capacity and may influence performance across tasks. Empirically, we observe that using smaller latent dimensions is beneficial on tasks such as Path-Star graph and Countdown.

Second, the design of the NextLat objective (e.g., stop-gradients, KL self-distillation, Smooth L1 loss) is guided largely by small-scale ablations in Section D and empirical intuition. It remains unclear whether multi-step supervision ($d > 1$) and KL token-level supervision are even necessary at larger model and data scales. More systematic studies are needed to better understand how these components interact. Third, due to computational constraints, we did not evaluate

against more recent or specialized MTP variants such as the one introduced in Liu et al. [2024]. Finally, we did not fully exploit the variable-length nature of NextLat’s speculative decoding. In our experiments, the draft length remained fixed throughout decoding for each prompt; we only varied the static draft length between 2 and 10 tokens to identify the configuration with the highest inference speedup. We leave the exploration of more creative adaptive-length speculative decoding strategies for NextLat to future work.

On the analysis side, we do not study the structure of the learned representations under NextLat, leaving open questions about how the method shapes latent spaces. In Section E, we also highlight several quirks observed during pretraining with NextLat, such as increases in Smooth L1 loss ($\mathcal{L}_{\text{next-h}}$) during training and differing loss trajectories across optimizers. These observations suggest that NextLat can be sensitive to optimization dynamics. Better understanding of how to scale and parameterize the NextLat objective remains an important direction for future work.

This work represents only a preliminary study of next-latent prediction, leaving many promising directions for future research. Since the method requires no architectural changes beyond a lightweight latent dynamics model for shaping representations, an interesting direction is to apply it as a post-hoc finetuning objective for pretrained transformers. This could potentially improve reasoning, planning, and world-modeling capabilities of existing models without retraining from scratch. Moreover, because NextLat effectively organizes latent representations with recurrent-like dynamics, an interesting question is whether transformers trained with NextLat are better suited for RL post-training, where value estimation (be it implicit or explicit) benefits from such recursive “Bellman-like” latent structure. Finally, it would be valuable to explore richer latent architectures, such as higher-dimensional or hierarchical belief states spanning multiple layers or tokens, which may further improve long-horizon reasoning and planning.

7 Conclusion

In this paper, we introduced Next-Latent Prediction (NextLat), a simple yet powerful framework that augments next-token training with self-supervised latent-space prediction, enabling transformers to learn belief-state representations. Theoretically, we show that the pressure to form concise latent summaries of past information sufficient to predict the future arises naturally from the NextLat objective. Empirically, NextLat yields more compact, predictive, and generalizable representations across tasks in world modeling, reasoning, planning, and language modeling—all without changing the transformer’s architecture, parallel training efficiency, or inference procedure. By reintroducing a recurrent inductive bias through self-predictive latent dynamics, NextLat unifies the inherent bias toward compact and temporally consistent representations of recurrent models with the scalability and parallelism of transformers. Our method is broadly applicable for training transformers in autoregressive sequence modeling domains. Looking ahead, we view NextLat as a step toward training objectives that endow autoregressive sequence models with simpler, more compact, and therefore more generalizable representations of complex systems.

Acknowledgements

We thank Jason Eisner, Jordan T. Ash, Pradeep Varakantham, Andrea Zanette, Michael C. Mozer, Ying Fan, and the MSR AI Frontiers team for their valuable discussions and support.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Kwangjun Ahn, Alex Lamb, and John Langford. Efficient joint prediction of multiple future tokens, 2025. URL <https://arxiv.org/abs/2503.21801>.
- Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son

- Nguyen, Cl  mentine Fourier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025. URL <https://arxiv.org/abs/2502.02737>.
- Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length generalization in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 38546–38556. Curran Associates, Inc., 2022.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- Gregor Bachmann and Vaishnavh Nagarajan. The pitfalls of next-token prediction. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 2296–2318. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/bachmann24a.html>.
- Randall Balestriero, Mark Ibrahim, Vlad Sobal, Ari Morcos, Shashank Shekhar, Tom Goldstein, Florian Bordes, Adrien Bardes, Gregoire Mialon, Yuandong Tian, Avi Schwarzschild, Andrew Gordon Wilson, Jonas Geiping, Quentin Garrido, Pierre Fernandez, Amir Bar, Hamed Pirsiavash, Yann LeCun, and Micah Goldblum. A cookbook of self-supervised learning, 2023. URL <https://arxiv.org/abs/2304.12210>.
- Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.
- Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred Warmuth. Classifying learnable geometric concepts with the vapnik-chervonenkis dimension. In *Proceedings of the eighteenth annual ACM symposium on Theory of computing*, pages 273–282, 1986.
- Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge (Jimmy) Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando De Freitas, Satinder Singh, and Tim Rock  tschel. Genie: generative interactive environments. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Roger C Conant and W Ross Ashby. Every good regulator of a system must be a model of that system. *International journal of systems science*, 1(2):89–97, 1970.
- Countdown. Countdown (game show), 2025. URL [https://en.wikipedia.org/wiki/Countdown_\(game_show\)](https://en.wikipedia.org/wiki/Countdown_(game_show)). [Online; accessed 12-October-2025].
- Kenneth James Williams Craik. *The nature of explanation*, volume 445. CUP Archive, 1967.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Fkckkr3ya8>.
- Ezra Edelman, Nikolaos Tsilivis, Benjamin L. Edelman, Eran Malach, and Surbhi Goel. The evolution of statistical induction heads: In-context learning markov chains. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet,

- J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 64273–64311. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/75b0edb869e2cd509d64d0e8ff446bc1-Paper-Conference.pdf.
- Ronen Eldan and Yuanzhi Li. Tinstories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.
- Hugging Face. Dataset card for bookcorpusopen, 2021. URL <https://huggingface.co/datasets/bookcorpusopen>.
- François Fleuret. The free transformer, 2025. URL <https://arxiv.org/abs/2510.17558>.
- Wikimedia Foundation. Wikimedia downloads. URL <https://dumps.wikimedia.org>.
- Bruce A Francis and Walter Murray Wonham. The internal model principle of control theory. *Automatica*, 12(5): 457–465, 1976.
- Karl Friston. The free-energy principle: a unified brain theory? *Nature reviews neuroscience*, 11(2):127–138, 2010.
- Samir Yitzhak Gadre, Georgios Smyrnis, Vaishaal Shankar, Suchin Gururangan, Mitchell Wortsman, Rulin Shao, Jean Mercat, Alex Fang, Jeffrey Li, Sedrick Keh, Rui Xin, Marianna Nezhurina, Igor Vasiljevic, Jenia Jitsev, Luca Soldaini, Alexandros G. Dimakis, Gabriel Ilharco, Pang Wei Koh, Shuran Song, Thomas Kollar, Yair Carmon, Achal Dave, Reinhard Heckel, Niklas Muennighoff, and Ludwig Schmidt. Language models scale reliably with over-training and on downstream tasks, 2024. URL <https://arxiv.org/abs/2403.08540>.
- Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D Goodman. Stream of search (sos): Learning to search in language. *arXiv preprint arXiv:2404.03683*, 2024.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 09–15 Jun 2019. URL <https://proceedings.mlr.press/v97/gelada19a.html>.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.
- Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. *arXiv preprint arXiv:2310.02226*, 2023.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces, 2022. URL <https://arxiv.org/abs/2111.00396>.
- Wes Gurnee and Max Tegmark. Language models represent space and time. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=jE8xbmvFin>.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3), 2018.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.

- Danijar Hafner, Timothy P. Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=0oabwyZbOu>.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. 640(8059):647–653, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-08744-2. URL <https://doi.org/10.1038/s41586-025-08744-2>.
- Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control, 2022. URL <https://arxiv.org/abs/2203.04955>.
- Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2023. URL <https://arxiv.org/abs/1606.08415>.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Edward S. Hu, Kwangjun Ahn, Qinghua Liu, Haoran Xu, Manan Tomar, Ada Langford, Jayden Teoh, Bryon Xu, David Yan, Dinesh Jayaraman, Alex Lamb, and John Langford. The belief state transformer, 2025. URL <https://arxiv.org/abs/2410.23506>.
- Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.
- Hai Huang, Yann LeCun, and Randall Balestriero. Llm-jepa: Large language models meet joint embedding predictive architectures. *arXiv preprint arXiv:2509.14252*, 2025.
- Philip Nicholas Johnson-Laird. *Mental models: Towards a cognitive science of language, inference, and consciousness*. Number 6. Harvard University Press, 1983.
- Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998.
- Andrej Karpathy. NanoGPT. <https://github.com/karpathy/nanoGPT>, 2022.
- Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Drrl2gcjzl>.
- John Langford and Robert Schapire. Tutorial on practical prediction theory for classification. *Journal of machine learning research*, 6(3), 2005.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023. URL <https://arxiv.org/abs/2211.17192>, 1(2), 2022.
- Kenneth Li, Aspen K Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Emergent world representations: Exploring a sequence model trained on a synthetic task. *ICLR*, 2023.
- Lihong Li, Thomas J Walsh, and Michael L Littman. Towards a unified theory of state abstraction for mdps. *AI&M*, 1(2):3, 2006.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024a.

- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 7421–7432, 2024b.
- Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model, 2024. URL <https://arxiv.org/abs/2403.19887>.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=De4FYqjFueZ>.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.
- Shuo Liu, Adria Mallol-Ragolta, Emilia Parada-Cabeleiro, Kun Qian, Xin Jing, Alexander Kathan, Bin Hu, and Bjoern W. Schuller. Audio self-supervised learning: A survey, 2022. URL <https://arxiv.org/abs/2203.01205>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Ashok Vardhan Makkuva, Marco Bondaschi, Adway Girish, Alliot Nagle, Martin Jaggi, Hyeji Kim, and Michael Gastpar. Attention with markov: A curious case of single-layer transformers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=SqZ0KY4qBD>.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.
- William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers, 2023. URL <https://arxiv.org/abs/2207.00729>.
- William Merrill, Ashish Sabharwal, and Noah A. Smith. Saturated transformers are constant-depth threshold circuits. *Transactions of the Association for Computational Linguistics*, 10:843–856, 2022. doi: 10.1162/tacl_a_00493. URL <https://aclanthology.org/2022.tacl-1.49/>.
- William Merrill, Jackson Petty, and Ashish Sabharwal. The illusion of state in state-space models, 2025. URL <https://arxiv.org/abs/2404.08819>.
- R Chris Miall and Daniel M Wolpert. Forward models for physiological motor control. *Neural networks*, 9(8): 1265–1279, 1996.
- Vaishnavh Nagarajan, Chen Henry Wu, Charles Ding, and Aditi Raghunathan. Roll the dice & look before you leap: Going beyond the creative limits of next-token prediction. *arXiv preprint arXiv:2504.15266*, 2025.
- Tianwei Ni, Benjamin Eysenbach, Erfan SeyedSalehi, Michel Ma, Clement Gehring, Aditya Mahajan, and Pierre-Luc Bacon. Bridging state and history representations: Understanding self-predictive RL. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=ms0VgzSGF2>.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 1525–1534, 2016.

- Jongho Park, Jaeseung Park, Zheyang Xiong, Nayoung Lee, Jaewoong Cho, Samet Oymak, Kangwook Lee, and Dimitris Papailiopoulos. Can mamba learn how to learn? a comparative study on in-context learning tasks, 2024. URL <https://arxiv.org/abs/2402.04248>.
- Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. Openwebmath: An open dataset of high-quality mathematical web text, 2023.
- Roma Patel and Ellie Pavlick. Mapping language models to grounded conceptual spaces. In *International conference on learning representations*, 2022.
- Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets, 2022. URL <https://arxiv.org/abs/2201.02177>.
- Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling, 2025. URL <https://arxiv.org/abs/2406.07522>.
- Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *2007 15th European signal processing conference*, pages 606–610. IEEE, 2007.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. Social iqa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 4463–4473, 2019.
- J. Schmidhuber. *Making the World Differentiable: On Using Self Supervised Fully Recurrent Neural Networks for Dynamic Reinforcement Learning and Planning in Non-Stationary Environments*. Forschungsberichte Künstliche Intelligenz. Inst. für Informatik, 1990.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- Max Schwarzer, Ankesh Anand, Rishab Goel, R Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations, 2021. URL <https://arxiv.org/abs/2007.05929>.
- Charlotte Striebel. Sufficient statistics in the optimum control of stochastic systems. *Journal of Mathematical Analysis and Applications*, 12(3):576–592, 1965.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023. URL <https://arxiv.org/abs/2104.09864>.
- Jayakumar Subramanian, Amit Sinha, Raihan Seraj, and Aditya Mahajan. Approximate information state for approximate planning and reinforcement learning in partially observed systems. *Journal of Machine Learning Research*, 23(12):1–83, 2022.
- Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160–163, 1991.
- Yunhao Tang, Zhaohan Daniel Guo, Pierre Harvey Richemond, Bernardo Ávila Pires, Yash Chandak, Rémi Munos, Mark Rowland, Mohammad Gheshlaghi Azar, Charline Le Lan, Clare Lyle, András György, Shantanu Thakoor, Will Dabney, Bilal Piot, Daniele Calandriello, and Michal Valko. Understanding self-predictive learning for reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org, 2023.

- Keyon Vafa, Justin Y Chen, Ashesh Rambachan, Jon Kleinberg, and Sendhil Mullainathan. Evaluating the world model implicit in a generative model. *Advances in Neural Information Processing Systems*, 37:26941–26975, 2024.
- Keyon Vafa, Peter G Chang, Ashesh Rambachan, and Sendhil Mullainathan. What has a foundation model found? using inductive bias to probe for world models. In *International Conference on Machine Learning*, pages 60727–60747. PMLR, 2025.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Johannes Welbl, Nelson F Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*, pages 94–106, 2017.
- Zhaofeng Wu, Linlu Qiu, Alexis Ross, Ekin Akyürek, Boyuan Chen, Bailin Wang, Najoung Kim, Jacob Andreas, and Yoon Kim. Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1819–1862, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.102. URL <https://aclanthology.org/2024.naacl-long.102/>.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/271db9922b8d1f4dd7aaef84ed5ac703-Paper-Conference.pdf.
- Jiacheng Ye, Jiahui Gao, Shansan Gong, Lin Zheng, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Beyond autoregression: Discrete diffusion for complex reasoning and planning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=NRYgUzSPZz>.
- Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. *Advances in neural information processing systems*, 34:25476–25488, 2021.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4791–4800, 2019.
- Amy Zhang, Rowan McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. *arXiv preprint arXiv:2006.10742*, 2020.
- Guanhua Zhang, Ricardo Dominguez-Olmedo, and Moritz Hardt. Train-before-test harmonizes language model rankings. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=ORv3SAzus1>.
- Kexin Zhang, Qingsong Wen, Chaoli Zhang, Rongyao Cai, Ming Jin, Yong Liu, James Zhang, Yuxuan Liang, Guansong Pang, Dongjin Song, and Shirui Pan. Self-supervised learning for time series analysis: Taxonomy, progress, and prospects, 2024. URL <https://arxiv.org/abs/2306.10125>.

A Belief States in Sequence Modeling

Recent work has introduced variants of sequence modeling architectures based on the principle of learning belief states, i.e., BST and JTP. We review these methods here. Let θ denote the parameters of a transformer-based model. Let $\mathbf{h}_{s:t}$ denote the hidden states produced by the transformer encoder for a token sequence $X_{s:t}$, where $s \leq t$. When we use the notation $\mathbf{h}_t$, it is shorthand for $\mathbf{h}_{1:t}$. The model’s output head produces a categorical distribution over the token vocabulary conditioned on some hidden state representation, i.e., $p_\theta(\cdot \mid \mathbf{h}_{s:t})$.

Belief State Transformer. The Belief State Transformer (BST; Hu et al. [2025]) learns *compact* belief states by jointly training a next-token predictor and a previous-token predictor across all possible prefix–suffix decompositions of a sequence, including cases where either the prefix or suffix is empty. Concretely, given a prefix $X_{1:t}$ and a suffix $X_{t+k:T}$ with $k \geq 1$, BST aims to minimize the cross-entropy loss

$$\mathcal{L}_{\text{BST}}(\theta) = \mathbb{E}_{t < T} \left[\underbrace{-\log p_\theta(X_{t+1} \mid \mathbf{h}_{1:t}, \mathbf{h}_{t+k:T})}_{\text{next-token prediction}} - \underbrace{\log p_\theta(X_{t+k-1} \mid \mathbf{h}_{1:t}, \mathbf{h}_{t+k:T})}_{\text{previous-token prediction}} \right], \quad (6)$$

where $\mathbf{h}_{1:t}$ and $\mathbf{h}_{t+k:T}$ are produced by separate transformers. This bidirectional training shapes the hidden representations of the BST into belief states.

Joint Multi-Token Prediction. Joint multi-token prediction (JTP; Ahn et al. [2025]) aims to learn the *joint* distribution over the next $d + 1$ tokens conditioned on $\mathbf{h}_t$, where d is the multi-step prediction horizon beyond the next token. Specifically, JTP minimizes the loss

$$\mathcal{L}_{\text{JTP}}(\theta; d, \lambda_{\text{MTP}}) = \mathbb{E}_{t < T} \left[\underbrace{-\log p_\theta(X_{t+1} \mid \mathbf{h}_t)}_{\text{next-token prediction}} - \lambda_{\text{MTP}} \cdot \frac{1}{d} \sum_{i=1}^d \underbrace{\log p_\theta(X_{t+i+1} \mid \text{Fetch}(\mathbf{h}_t, X_{t+1:t+i}))}_{\text{joint multi-token prediction}} \right], \quad (7)$$

where an additional module $\text{Fetch}(\mathbf{h}_t, X_{t+1:t+i})$ is used to create an embedding combining the teacher-forced tokens $X_{t+1:t+i}$ with $\mathbf{h}_t$ and $\lambda_{\text{MTP}} > 0$ balances next- and multi-token prediction losses. Although Ahn et al. [2025] suggest that their method learn “short-horizon belief states”, they do not formally define the conditions under which this occurs. To understand how JTP learns belief states, we start by defining a *k-observable* system.

Definition A.1 (*k-observability for sequences*). A system is *k-observable* if for any two sequences $H = X_{1:t}$ and $H' = X_{1:j}$ that induce the same joint distribution over the next- k tokens, i.e., $\mathbb{P}(X_{t+1:t+k} \mid H) = \mathbb{P}(X_{t+1:t+k} \mid H')$, it follows that their *full-horizon* conditionals match:

$$\mathbb{P}(X_{t+1:T} \mid H) = \mathbb{P}(X_{t+1:T} \mid H'). \quad (8)$$

In other words, the distribution of all future observations is determined by the distribution of the next k observations.

Proposition A.2 (JTP forms belief states in *k-observable* systems). Assume the system is *k-observable* and let $k = d + 1$. Suppose the joint multi-token prediction model recovers the true joint next- k conditional, i.e. $p_\theta(X_{t+1} \mid \mathbf{h}_t)p_\theta(X_{t+2} \mid \mathbf{h}_t, X_{t+1}) \dots p_\theta(X_{t+k} \mid \mathbf{h}_t, X_{t+1:t+k-1}) = \mathbb{P}(X_{t+1:t+k} \mid X_{1:t})$ a.s. for all t , then $\mathbf{h}_t$ is a belief state for $X_{1:t}$.

Proof. By *k-observability* (Theorem A.1), there exists a measurable map G taking the joint next- k conditional distribution to the full-horizon conditional:

$$\mathbb{P}(X_{t+1:T} \mid X_{1:t}) = G\left(\mathbb{P}(X_{t+1:t+k} \mid X_{1:t})\right).$$

By the premise that JTP recovers the true next- k joint, conditioning on $\mathbf{h}_T$ is equivalent to conditioning on $X_{1:t}$ for all bounded measurable functionals of the future. Hence $\mathbf{h}_t$ is a belief state for all $t < T$. $\square$

Intuitively, if all possible futures can be distinguished by the next k tokens, then a JTP model that accurately predicts the joint next- k distribution would encode all information necessary to distinguish future trajectories. Note that both next-token prediction and multi-token prediction do not guarantee belief state representations (see Hu et al. [2025]).

B Formal Proof of Theorem 3.2

The proof follows the intuition illustrated below. Optimizing for next-token and transition consistency (Equations 1 and 2) ensures the existence of measurable maps p_θ and p_ψ that allow recursive decoding of future tokens:

$$\mathbf{h}_t \xrightarrow[\text{decode token}]{p_\theta} X_{t+1} \xrightarrow[\text{update state}]{p_\psi} \mathbf{h}_{t+1} \xrightarrow[\text{decode token}]{p_\theta} X_{t+2} \xrightarrow[\text{update state}]{p_\psi} \mathbf{h}_{t+2} \cdots \xrightarrow{p_\theta} X_T.$$

Proof. A formal proof proceeds by backward induction on t . For the base case $t = T - 1$, the claim follows directly from Equation 1, since $\mathbf{h}_{T-1}$ suffices to predict X_T .

Now assume $\mathbf{h}_{k+1}$ is a belief state for $X_{1:k+1}$. By Theorem 3.1, this implies that $X_{k+2:T}$ is conditionally independent of $X_{1:k+1}$ given $\mathbf{h}_{k+1}$. We will show that $\mathbf{h}_k$ is also a belief state for $X_{1:k}$. From $\mathbf{h}_k$, one can generate

$$\begin{aligned} X_{k+1} &\sim p_\theta(\cdot | \mathbf{h}_k), \\ \mathbf{h}_{k+1} &\sim p_\psi(\cdot | \mathbf{h}_k, X_{k+1}). \end{aligned}$$

By next-token and transition consistency (Equations 1 and 2), we have

$$\begin{aligned} \mathbb{P}(X_{k+1:T} | \mathbf{h}_k) &= \mathbb{P}(X_{k+2:T} | X_{k+1}, \mathbf{h}_k) \underbrace{\mathbb{P}(X_{k+1} | \mathbf{h}_k)}_{\text{Equation 1}} \\ &= \mathbb{P}(X_{k+2:T} | X_{k+1}, \mathbf{h}_k) \mathbb{P}(X_{k+1} | X_{1:k}) \\ &= \left[\int \mathbb{P}(X_{k+2:T}, \mathbf{h}_{k+1} | X_{k+1}, \mathbf{h}_k) d\mathbf{h}_{k+1} \right] \mathbb{P}(X_{k+1} | X_{1:k}) \\ &= \left[\int \underbrace{\mathbb{P}(X_{k+2:T} | \mathbf{h}_{k+1}, X_{k+1}, \mathbf{h}_k)}_{\text{Equation 2}} \underbrace{\mathbb{P}(\mathbf{h}_{k+1} | X_{k+1}, \mathbf{h}_k)}_{\text{Equation 2}} d\mathbf{h}_{k+1} \right] \mathbb{P}(X_{k+1} | X_{1:k}) \\ &= \left[\int \underbrace{\mathbb{P}(X_{k+2:T} | \mathbf{h}_{k+1})}_{\text{Induction hypothesis}} \mathbb{P}(\mathbf{h}_{k+1} | X_{1:k+1}) d\mathbf{h}_{k+1} \right] \mathbb{P}(X_{k+1} | X_{1:k}) \\ &= \mathbb{P}(X_{k+2:T} | X_{1:k+1}) \mathbb{P}(X_{k+1} | X_{1:k}) = \mathbb{P}(X_{k+1:T} | X_{1:k}). \end{aligned}$$

This proves that $\mathbf{h}_k$ is also a belief state. □

C More Details on NextLat Implementation

We parameterize the latent transition model p_ψ with a three-layer MLP using GELU [Hendrycks and Gimpel, 2023] activations. The latent transition model takes as input the layer-normalized [Ba et al., 2016] concatenation of the current hidden state $\mathbf{h}_t$ and next-token embedding X_{t+1} , and outputs a *delta* update applied via residual connection:

$$\hat{\mathbf{h}}_{t+1} = p_\psi(\mathbf{h}_t, X_{t+1}) = f_\psi(\mathbf{h}_t, X_{t+1}) + \mathbf{h}_t \quad (9)$$

where $f_\psi(\cdot)$ predicts the modification to $\mathbf{h}_t$ (see Figure 11). This paper aims to demonstrate that NextLat yields significant performance gains even with a **simple MLP latent transition**. We foresee even better performances with more sophisticated latent transition model architectures, but we leave that exploration to future work.

Following standard training, we mask token-level losses (i.e., $\mathcal{L}_{\text{next-token}}$ and $\mathcal{L}_{\text{KL}}$) corresponding to context or prompt tokens. However, we do not apply masking for $\mathcal{L}_{\text{next-h}}$ on context tokens, ensuring that belief state representations develop even during context processing. When using sequence packing (e.g., for Manhattan, TinyStories, and FineWeb-Edu), we mask $\mathcal{L}_{\text{KL}}$ and $\mathcal{L}_{\text{next-h}}$ terms that cross document boundaries.

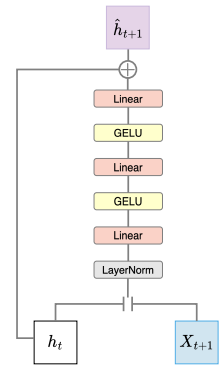


Figure 11: Illustration of the latent transition model p_ψ .

Algorithm 1 Pseudocode for NextLat’s loss function in PyTorch syntax. The loss functions `cross_entropy_loss()`, `smooth_l1_loss()`, and `KL_loss()` are implemented externally.

```

1 import torch.nn as nn
2 from copy import deepcopy
3
4 def loss(batch, targets):
5     # batch:(B,T), targets: (B,T)
6     # embedding dimension = D, vocabulary size = V
7     batch_size, seq_len = batch.shape
8
9     # Encode sequences into latent states
10    hidden_states = Transformer(batch) # (B,T,D)
11
12    # Compute next-token loss
13    logits_post = Output_Head(hidden_states) # (B,T,V)
14    loss_next = cross_entropy_loss(logits_post, targets)
15
16    next_tokens = batch
17    next_states = hidden_states
18    current_states = hidden_states
19    loss_next_h = 0
20    loss_kl = 0
21
22    # Recursive multi-step predictions, default multi_step_horizon=1;
23    # only 1-step prediction
24    for _ in range(multi_step_horizon):
25        # Shift hidden states back by 1
26        current_states = current_states[:, :-1]
27        # Shift next tokens and hidden state targets forward by 1
28        next_tokens = next_tokens[:, 1:]
29        next_states = next_states[:, 1:]
30        logits_post = logits_post[:, 1:]
31        # Predict next hidden state using latent dynamics model
32        # i.e.,  $(h_t, x_{t+1}) \rightarrow \hat{h}_{t+1}$ 
33        pred_next_states = Latent_Dynamics(current_states, next_tokens) # (B,T,D)
34        # Compute next-hidden loss using detached next states as targets
35        # i.e.,  $\|h_{t+1} - \hat{h}_{t+1}\|^2$ 
36        loss_next_h += smooth_l1_loss(pred_next_states, next_states.detach())
37
38        # Compute KL loss using detached output head
39        # i.e.,  $KL[p(h_{t+1}) || p(\hat{h}_{t+1})]$ 
40        logits_prior = deepcopy(Output_Head)(pred_next_states) # (B,T,V)
41        loss_kl += KL_loss(logits_post.detach(), logits_prior) # detach posterior
42
43        current_states = pred_next_states
44
45    loss_next_h = loss_next_h / multi_step_horizon
46    loss_kl = loss_kl / multi_step_horizon
47    # overall NextLat loss
48    return loss_next + next_h_lambda * loss_next_h + kl_lambda * loss_kl

```

D Ablation Studies

In this section, we ablate the key design choices of NextLat. Specifically, we study the effects of the KL ($\mathcal{L}_{\text{KL}}$) and Smooth L1 ($\mathcal{L}_{\text{next-h}}$) losses, as well as the use of stop-gradients on the target ($\text{sg}[\mathbf{h}_{t+i}]$) in the Smooth L1 loss (Equation 3) and on the output head ($p_{\theta}^{\text{sg}}(\cdot)$) in the KL loss (Equation 4). We focus our investigations primarily on TinyStories, analyzing how these design choices affect the predictive quality of learned hidden states under linear probing and validation loss behavior.

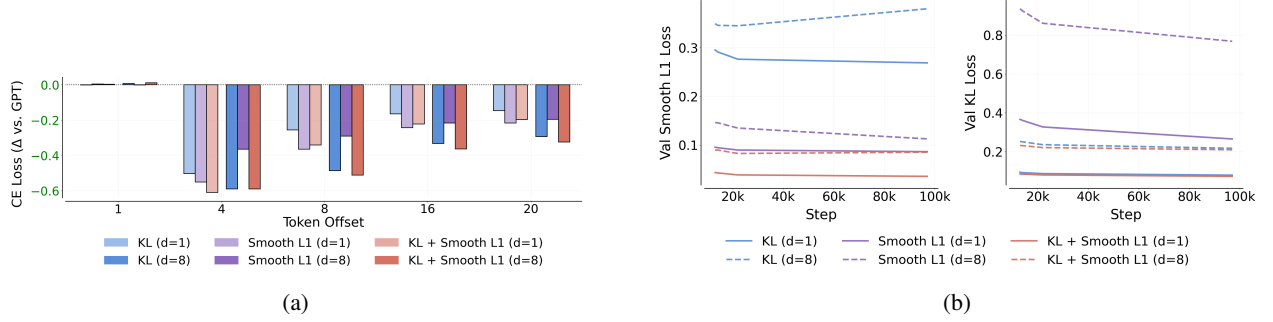


Figure 12: **KL and Smooth L1 Loss Ablations:** (a) TinyStories probe performance under ablations of the KL and Smooth L1 losses. (b) TinyStories validation Smooth L1 and KL losses.

We first isolate the effects of the KL and Smooth L1 losses in NextLat. Note that $\mathcal{L}_{\text{next-h}}$ is required for belief-state convergence, while $\mathcal{L}_{\text{KL}}$ serves as a complementary supervision signal. In Figure 12a, we observe that at $d = 1$, using Smooth L1 loss alone achieves the strongest probe performance 20 tokens ahead relative to GPT. At $d = 8$, however, the combined KL + Smooth L1 objective (i.e., the full NextLat design) performs best, suggesting that the Smooth L1 loss is critical for learning long-range predictive representations, while the KL loss becomes increasingly beneficial at larger horizons. Furthermore, Figure 12b shows that the combined KL + Smooth L1 objective achieves lower validation KL and Smooth L1 losses than optimizing either objective alone. This confirms that the two losses are complementary and provide mutually beneficial supervision.

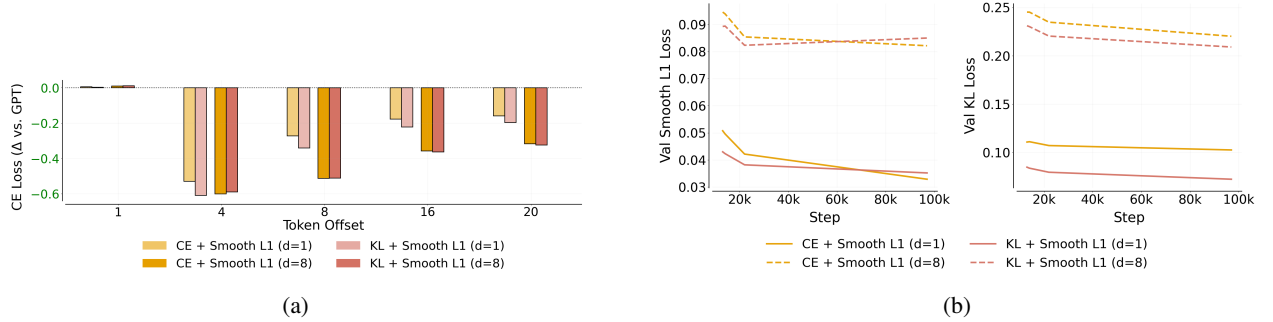


Figure 13: **KL and CE Loss Ablations:** (a) TinyStories probe performance when replacing the KL loss in NextLat with a CE loss. (b) TinyStories validation Smooth L1 and KL losses.

Next, we compare the KL loss against cross-entropy (CE) as the complementary token-space supervision signal. Note that the CE loss corresponds exactly to the multi-token prediction objective used in JTP and MTP. Unlike the KL loss, which performs self-distillation by matching the predicted token distribution to the model’s own softmax distribution under the target hidden state (see Equation 4), the CE loss directly supervises the predicted latent state using the ground-truth next token. As shown in Figure 13a, the original NextLat objective (KL + Smooth L1) achieves substantially better probe performance than CE + Smooth L1 at $d = 1$, though the gap becomes marginal at $d = 8$. Figure 13b shows that across both $d = 1$ and $d = 8$, KL + Smooth L1 achieves lower validation KL loss, but slightly

worse Smooth L1 loss, than CE + Smooth L1. Overall, these results suggest that at shallow multi-step prediction horizons, the dense distribution-level supervision provided by KL matching induces more predictive representations than CE. However, this advantage appears to diminish at larger horizons. It is also important to note that the KL loss incurs higher memory overhead than CE loss. This is because KL requires materializing full logits and softmax distributions over the vocabulary, whereas modern fused CE implementations only need to materialize probabilities at the target token index. More systematic studies are needed, especially at larger model scales, to determine whether KL or CE is preferable as a complementary supervision signal. For the scope of this work, we stick to using KL loss in all experiments.

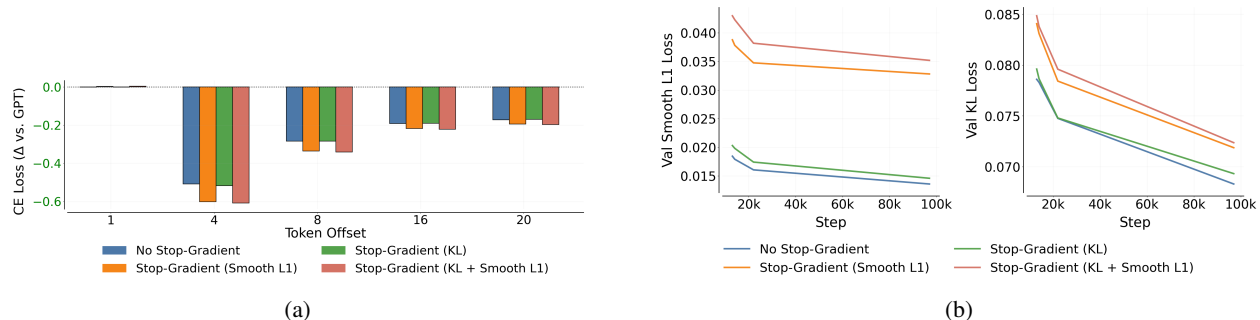


Figure 14: **Stop-Gradient Ablations:** (a) TinyStories probe performance under different stop-gradient configurations. (b) TinyStories validation Smooth L1 and KL losses. Note that all models here use both KL and Smooth L1 losses and differ only in where stop-gradients are applied in those loss components. “Stop-Gradient (Smooth L1)” applies stop-gradient only to the target hidden state in the Smooth L1 loss, “Stop-Gradient (KL)” applies stop-gradient only to the output head in the KL loss, and “Stop-Gradient (KL + Smooth L1)” applies stop-gradients to both components. All models use $d = 1$ here.

Next, we study the use of stop-gradients in Figure 14 with $d = 1$. While Figure 14b shows that removing the stop-gradient on the Smooth L1 loss improves validation Smooth L1 and KL losses slightly (~ 0.02), it does not improve the predictive quality of the learned representations. As shown in Figure 14a, applying stop-gradients to both the target in the Smooth L1 loss and the output head ($p_{\theta}^{\text{sg}}(\cdot)$) in the KL loss yields the best probing performance across all token offsets. Stop-gradients also improve efficiency by reducing the number of backward passes through the transformer. In particular, the KL loss introduces an additional forward and backward pass through the output head, which can be computationally expensive in language models due to large vocabulary sizes⁸. Removing this extra backward pass significantly improves both speed and memory efficiency. Empirically, on tasks like Countdown and Path-Star graph, we also observe that the stop-gradient (especially on the Smooth L1 loss) is essentially for high accuracy.

Overall, these ablations suggest that NextLat’s effectiveness arises from the combination of Smooth L1 and KL losses together with carefully placed stop-gradients, all of which contribute to learning predictive belief-state representations in transformers.

E NextLat Pretraining Quirks

In this section, we expose some optimization quirks that we faced during pretraining with NextLat. First, Figure 15a shows vastly different Smooth L1 loss trajectories when using Muon [Jordan et al., 2024] versus AdamW [Loshchilov and Hutter, 2019] optimizer under the same learning rate schedule (using the Muon update scaling rule proposed by Liu et al. [2025]). One possible explanation is that Muon’s orthogonalized update rule enables more efficient latent representation shaping and next-latent prediction optimization. However, Figure 15b shows that AdamW achieves better KL loss optimization than Muon.

⁸Note that multi-token prediction models also incur an additional forward+backward pass through the output head for each extra token prediction.

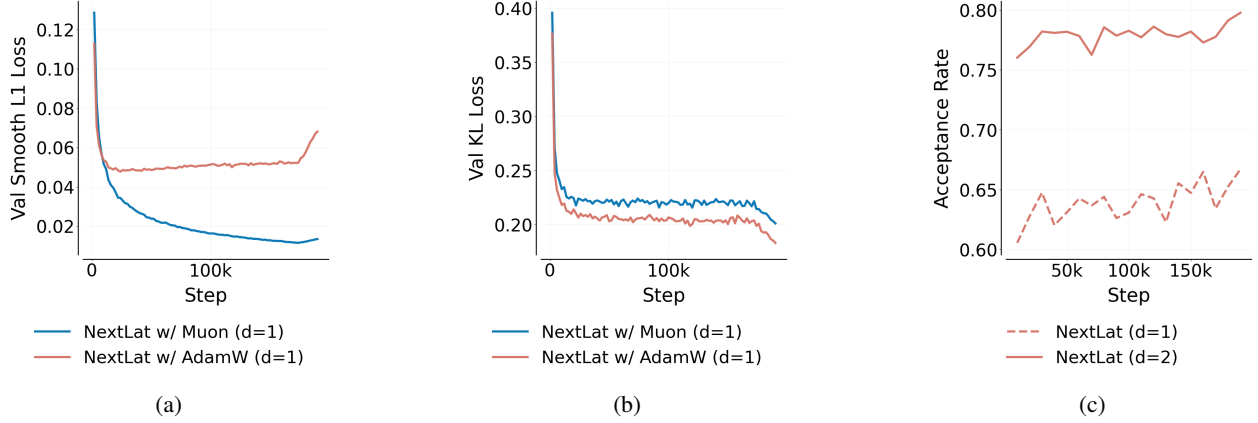


Figure 15: **FineWeb-Edu Pretraining Curves:** (a) Validation smooth L1 loss during training. (b) Validation KL loss during training. (c) Speculative decoding acceptance rate over a 5-token lookahead horizon (excluding the next-token prediction).

Importantly, both optimizers exhibit **an increase in Smooth L1 loss near the end of training** corresponding to the learning rate cooldown stage, though the effect is substantially more pronounced for AdamW. The cause of this behavior remains unclear. We explored several potential fixes to no avail, including:

1. replacing Smooth L1 with MSE loss,
2. removing stop-gradients from $\mathcal{L}_{\text{next-h}}$,
3. tuning loss coefficients $\lambda_{\text{next-h}}$ and λ_{KL} ,
4. disabling learning rate decay for the latent transition model, and
5. disabling weight decay for the latent transition model.

We leave a deeper investigation of this phenomenon to future work. To our reassurance, the increase in smooth L1 loss does not appear to degrade the coherence of the latent transition model. As shown in Figure 15c, the speculative decoding acceptance rate over a lookahead horizon of 5 tokens continues to improve throughout training, even during the phase where the smooth L1 loss increases under AdamW. This suggests that the rise in smooth L1 loss may reflect changes in the scale or geometry of the latent states, rather than degradation in the token-level coherence of the latent transition dynamics.

On a side note, we observed gradient norm spikes that led to training instability when training MTP with Muon, which unfortunately limited our pretraining analysis to AdamW (despite Muon showing a more favorable learning trajectory with NextLat).

F Extra Experiment Details

In this section, we provide additional experimental setup details and supplementary results for the experiments in the main body. All experiments were run on NVIDIA RTX A5000, NVIDIA H100 NVL, and NVIDIA B200 GPUs. Table 5 summarizes the training, model, and NextLat hyperparameters across all experimental domains.

The hyperparameters reported in Table 5 were chosen through a small-scale search, exploring $\lambda_{\text{next-h}} \in \{1.0, 2.0\}$ and $\lambda_{\text{KL}} \in \{0.1, 1.0\}$, guided by empirical observations and intuition. Encouragingly, we find that NextLat performs robustly over a wide range of settings. In particular, λ_{KL} requires minimal tuning; $\lambda_{\text{KL}} = 1.0$ works decently across all tasks. After all, it primarily serves as a complementary alignment objective. Likewise, $\lambda_{\text{next-h}} = 1.0$ is effective in most cases, though slightly higher values (e.g., $\lambda_{\text{next-h}} = 2.0$) was beneficial when the next-latent regression loss is of much smaller scale than the token-level losses (i.e., $\mathcal{L}_{\text{next-token}}$ and $\mathcal{L}_{\text{KL}}$).

For MTP and JTP, we sweep multi-token prediction loss weights $\lambda_{\text{MTP}} \in \{0.1, 0.2, 0.4, 0.6, 0.8, 1.0\}$ to ensure fair

	Manhattan	Countdown	Path-Star	TinyStories	A ₅	FineWeb-Edu
Training Parameters						
Steps	400k (6 epochs)	100k (204 epochs)	20k (51 epochs)	100k (8.5 epochs)	400k (512 epochs)	191k (100B tokens)
Batch Size	256	1024	512	256	1024	512
Learning Rate	1e-4	3e-4	5e-4	3e-4	1e-4	4e-4
Learning Rate Schedule	Constant	Constant	Constant	Constant	Constant	WSD
Weight Decay	0.01	0.1	0.1	0.1	0.01	0.1
Clip Gradient Norm	-	1	100	1	1	1
Optimizer	AdamW [Loshchilov and Hutter, 2019] with $\beta_1 = 0.9$, $\beta_2 = 0.95$					
Model Parameters						
Layers	48	12	12	8	2	22
Heads	8	12	6	8	8	16
Hidden Dimension	384	768	384	768	512	2048
NextLat Parameters						
$\lambda_{\text{next-h}}$	1.0	2.0	1.0	1.0	1.0	1.0
λ_{KL}	0.1	1.0	1.0	1.0	0.0	1.0
p_ψ MLP Hidden Dimension	1536	768	384	2048	1024	6528
p_ψ MLP Layers	3	3	3	3	3	3

Table 5: Training, Model, and NextLat hyperparameters across all benchmarks.

baseline comparisons. For FineWeb-Edu pretraining, extensive hyperparameter tuning is computationally prohibitive, so we use uniform loss weights across all methods (i.e., $\lambda_{\text{next-h}} = \lambda_{\text{KL}} = \lambda_{\text{MTP}} = 1.0$) to provide the fairest comparison possible under our compute constraints.

F.1 Manhattan Taxi Rides

Here, we provide additional details on our training and evaluation setups for the Manhattan Taxi Rides benchmark and clarify key differences from the original study of Vafa et al. [2024].

Training. Since this task inherently requires state tracking (i.e., tracking position within Manhattan), and increasing model depth is known to benefit transformers on such tasks [Merrill et al., 2025], we employ 48-layer transformers with 384 hidden dimensions and 8 attention heads (88M parameters). This differs from Vafa et al. [2024], which used 12 layers, 768 hidden dimensions, and 12 heads for their smaller transformer variant. We found that increasing model depth yielded substantial performance gains, whereas increasing hidden dimensionality offered negligible improvement. As shown in Table 1, the effective latent rank of our models is substantially smaller than 384, suggesting that large hidden dimensions are unnecessary.

Unlike the original study, which trained models for only one epoch, our models are trained for six epochs, as we observed that performances generally do not converge within a single epoch. This also helps rule out potential “grokking” phenomena [Power et al., 2022], where generalization improves only after extended periods of overfitting. To enable longer training without substantially increasing runtime, we apply sequence packing, i.e., concatenating multiple sequences into longer ones while masking cross-sequence attention. This enables efficient utilization of GPU memory and computation. Most models complete six training epochs in under three days on a single NVIDIA H100 NVL GPU, except MTP, which has substantially more parameters than the others.

Evaluation. Models are trained on random traversals of length 100 connected pickup and dropoff intersections. This task inherently requires not only state tracking but also planning, as models must reason over possible future paths to generate valid 100-step trajectories that reach the destination while avoiding dead ends, i.e., road segments disconnected from the goal due to the one-way streets. Random traversals rarely correspond to true shortest paths and do not provide an inductive bias toward learning the shortest path algorithm, which relies on dynamic programming. Consequently, unlike Vafa et al. [2024], who evaluated pairs with shortest paths of up to 100 steps, we limit evaluation pairs to paths of up to 50 steps. This adjustment ensures that evaluation pairs do not demand long-horizon planning beyond the training distribution, which might otherwise force the model to produce forced predictions to compensate for planning failure.

This also ensures that inconsistencies in a model’s internal map are more reflective of world-model incoherence rather than artifacts of long-horizon planning limitations. These evaluation pairs are used to generate Figure 3 and to compute the sequence compression and detour robustness metrics, following the procedure of Vafa et al. [2024].

For the effective latent rank, we pass a batch of 256 sequences (each of length 256) through the model to obtain the hidden state matrix. Singular values smaller than $1e-12$ are discarded, and the effective rank is then computed following Roy and Vetterli [2007]. For GPT and NextLat, we use the final-layer hidden states. For JTP, we extract the hidden states immediately before the self-attention module in the Fetch head (see Equations 4–5 in Ahn et al. [2025]). For MTP, we use the output of the next-token prediction head to compute the effective rank.

F.2 Countdown

We largely follow Gandhi et al. [2024] for the Countdown training and evaluation setup. Each problem consists of four input numbers and a solution sequence comprising three equations, consistent with prior work [Gandhi et al., 2024, Ye et al., 2025]. A training example is formatted as

$$\underbrace{14, 83, 88, 91}_{\text{inputs}}, \overbrace{23}^{\text{target}} \mid \underbrace{83 - 14 = 69, 91 - 88 = 3, 69/3 = 23}_{\text{solution}}$$

where the first four numbers are the inputs, the fifth is the target number, and the pipe symbol “|” separates the input prompt from the solution. During training, loss values corresponding to input prompt are masked out.

Previous studies involving the Countdown benchmark used pretrained GPT-2 byte-pair encoding tokenizers, which do not necessarily tokenize multi-digit numbers as single units. In contrast, we construct a custom tokenizer that assigns each integer from 1 to 10,000 to a unique token, ensuring that every number in the sequence is represented atomically. The arithmetic operators and delimiters, i.e., $\{ |, +, -, \times, \div \}$, are each assigned their own token indices. Due to the large branching factor of the Countdown problem, we insert eight pipe symbols (“|”) between the input and the solution as pause tokens [Goyal et al., 2023], allowing the model additional computation steps to plan before generating its answer.

F.3 Path-Star

Our Path-Star data preparation, training, and evaluation follow Bachmann and Nagarajan [2024], except that we increase the weight decay to 0.1, which we found helpful for stable convergence and higher solve rates in the multi-step prediction methods (i.e., MTP, JTP, and NextLat). We evaluate each model’s ability to generate the correct arm on 20k held-out test instances. Unlike Hu et al. [2025] and Ahn et al. [2025] which generate a fresh set of graphs every batch, we adopt the original, more challenging setup of Bachmann and Nagarajan [2024], which uses a fixed sample size of 200k and node values sampled from $N = 100$. This difference accounts for the performance gap observed in the BST and JTP baselines in Figure 6. The Path-Star experiment is designed to expose the myopic behavior of teacher-forced next-token prediction, which can encourage models to exploit superficial regularities—an effect referred to as the *Clever Hans cheat* [Bachmann and Nagarajan, 2024]. Because the task’s sample space grows exponentially with graph size, identifying the correct algorithm that generalizes across all graph instances is highly nontrivial. While not conclusive, our results suggest that latent-space prediction and the inductive bias toward compressing history into belief states promote better discovery of generalizable solutions in data-constrained settings.

F.4 TinyStories

Our TinyStories setup follows exactly Hu et al. [2025]. After pretraining on TinyStories, we train linear (one-layer) probes on the hidden states of the frozen transformer models for an additional 20k steps on the same dataset. All probe training hyperparameters (e.g., learning rate, batch size) match those used during pretraining (see Table 5).

For GPT, JTP, and NextLat, the choice of hidden states follows the setup used for measuring effective rank in the Manhattan taxi rides task, as described in Section F.1. For BST, we use the final-layer hidden states of the forward

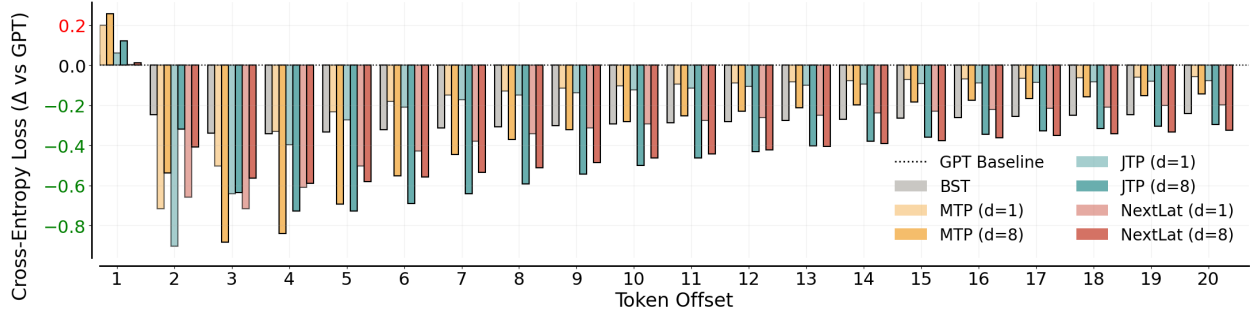


Figure 16: Full plot version of Figure 8 that shows probe performance across all 20 tokens offsets.

transformer encoder. For MTP [Gloeckle et al., 2024], we use the output of the shared transformer trunk, i.e., the hidden state before it branches into separate transformer heads for multi-token prediction, as this final shared representation contains the most predictive information about future tokens.

F.5 A_5 State Tracking

The group A_5 consists of the even permutations of 5 elements. Intuitively, each token in the sequence represents a permutation operation that rearranges the five elements, and after each operation the model must output the resulting arrangement (i.e., state). The task is therefore fundamentally a *state-tracking* problem: the model must maintain and update an internal representation of the current state as new permutation operations are sequentially composed.

Our training setup follows Merrill et al. [2025]. We trained the models on 1 million unique 12-token sequences over the group A_5 . We then evaluated their length generalization capabilities on approximately 100k 36-token sequences. For NextLat training, we used **one-step supervision** ($d = 1$) and optimized only the regression objective ($\lambda_{\text{next-h}} = 1$), without token-level supervision ($\lambda_{\text{KL}} = 0$). This ensures that any observed length generalization of the latent dynamics model (RNN) arises purely from faithful next-latent prediction rather than auxiliary multi-token prediction signals.

We also experimented with transformers without positional embeddings (NoPE), motivated by prior work suggesting improved length generalization [Kazemnejad et al., 2023]. However, we found that NoPE degraded performance in this setting, so all experiments use Rotary Position Embeddings (RoPE; Su et al. [2023]). Unlike RNNs, whose hidden states evolve sequentially and therefore implicitly encode token order, transformers process sequences in parallel and rely heavily on positional embeddings to represent token order information during training. Interestingly, despite positional embeddings being explicitly included in the transformer hidden states, the latent dynamics model trained on one-step latent transitions derived from the transformer still generalizes beyond the training sequence length. This suggests that the learned transition dynamics is robust to the transformer’s positional encodings.

F.6 FineWeb-Edu Pretraining

Following standard practices, all models are optimized with AdamW using a peak learning rate of $4e-4$, weight decay of 0.1, and gradient clipping at 1.0. We use a global batch size of 500M tokens and a sequence length of 1024 tokens. The learning rate follows a Warmup-Stable-Decay (WSD) schedule [Hu et al., 2024], consisting of a 1B-token linear warmup followed by a 10B-token linear decay phase. All models use the GPT-2 tokenizer with a vocabulary size of 50,257.

We evaluated the zero-shot accuracy of the pretrained models on nine standard language modeling benchmarks: Wikitext (Wiki; Merity et al. [2016]), LAMBADA (LAMB.; Paperno et al. [2016]) (standard version), PIQA [Bisk et al., 2020], HellaSwag (HellaS.; Zellers et al. [2019]), WinoGrande (Wino.; Sakaguchi et al. [2021]), ARC-easy (ARC-e) and ARC-challenge (ARC-c) [Clark et al., 2018], Social IQa (SIQA; Sap et al. [2019]), and SciQ [Welbl et al., 2017]. We also evaluate the self-speculative decoding performance of JTP, MTP, and NextLat across Wikipedia [Foundation],

Books (BookCorpusOpen; Face [2021]), Code (Stack-Edu; Allal et al. [2025]), and Math (OpenWebMath; Paster et al. [2023]) domains. To do so, we sample 1024 prompts of length 512 tokens from each dataset and generate 512-token continuations using the speculative sampling algorithm of Leviathan et al. [2022]. We then measured the speedup in inference of each model using self-speculative decoding compared to naive autoregressive sampling from the transformer, measured on 8×NVIDIA B200 GPUs.

F.6.1 Results with MTP/JTP ($d=4$)

Model	FW-Edu ppl ↓	Wiki. ppl ↓	LAMB. ppl ↓	LAMB. acc ↑	PIQA acc ↑	HellaS. acc ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc ↑	SIQA acc ↑	SciQ acc ↑	Avg.
GPT	10.52	17.93	20.26	42.07	73.45	58.79	<u>60.46</u>	68.18	39.16	42.32	86.10	58.82
JTP ($d=1$)	11.08	19.28	21.88	41.35	74.92	57.43	58.64	68.73	39.25	42.99	87.30	<u>58.83</u>
JTP ($d=2$)	11.18	19.60	22.11	41.37	73.34	56.84	59.98	68.86	38.57	43.35	86.70	58.63
JTP ($d=4$)	11.29	20.45	20.65	41.90	73.45	56.58	57.70	69.73	<u>39.68</u>	42.53	88.50	58.76
MTP ($d=1$)	10.90	18.82	20.23	41.26	<u>74.32</u>	58.05	60.54	68.52	38.91	42.84	85.40	58.76
MTP ($d=2$)	11.00	18.61	<u>18.34</u>	<u>43.43</u>	72.80	57.92	59.35	68.35	39.08	41.97	86.60	58.69
MTP ($d=4$)	11.10	18.97	22.75	40.69	73.72	57.39	58.33	70.20	39.51	42.12	85.90	58.48
NextLat ($d=1$)	<u>10.83</u>	<u>18.39</u>	19.77	41.08	73.07	<u>58.35</u>	59.27	69.65	<u>39.68</u>	43.24	86.00	58.79
NextLat ($d=2$)	10.88	18.44	17.83	43.86	73.61	57.79	59.20	<u>69.74</u>	40.10	41.91	<u>87.50</u>	59.21

Table 6: Downstream language modeling evaluation on 1.3B-parameter models trained on 100B FineWeb-Edu tokens. Best scores are in bold and second-best are underlined.

In this section, we extend our comparisons to include JTP and MTP trained with larger horizons ($d = 4$). Note that the training cost of JTP and MTP increases substantially with larger d (see Table 4), making JTP ($d = 4$) and MTP ($d = 4$) significantly more expensive to train than NextLat ($d = 1, 2$).

Table 6 shows that increasing the multi-token prediction horizon to $d = 4$ still does not yield meaningful improvements on multiple-choice benchmarks; NextLat ($d = 2$) continues to achieve the best average accuracy overall. Table 7 shows that even when JTP and MTP are trained to be able to draft more tokens (i.e., 4 tokens ahead), they still fail to surpass the variable-length speculative decoding performance of NextLat across most domains, with the exception of the Code domain. Finally, Figure 17 shows that, on the FineWeb-Edu validation set, increasing the training horizon to $d = 4$ yields only modest speedup gains for JTP and MTP. Their speedup curves saturate earlier and remain below that of NextLat ($d = 2$), which continues to improve with longer draft lengths and achieves the highest overall speedup.

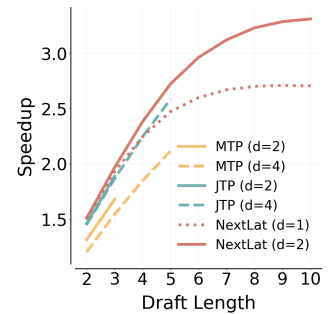


Figure 17: Inference speedup ratio on the FineWeb-Edu validation set.

Model	Wikipedia		Books		Code		Math	
	Speedup	Accepted Tokens	Speedup	Accepted Tokens	Speedup	Accepted Tokens	Speedup	Accepted Tokens
JTP ($d=1$)	1.46	0.96	1.47	0.97	1.47	0.98	1.46	0.97
JTP ($d=2$)	1.88	1.84	1.90	1.89	1.88	1.85	1.89	1.86
JTP ($d=4$)	2.58	3.32	2.62	3.43	2.61	<u>3.43</u>	<u>2.42</u>	3.02
MTP ($d=1$)	1.38	0.91	1.39	0.95	1.40	0.97	1.39	0.95
MTP ($d=2$)	1.68	1.72	1.72	1.83	1.75	1.91	1.72	1.84
MTP ($d=4$)	2.10	3.04	2.25	3.44	2.32	3.68	2.25	<u>3.46</u>
NextLat ($d=1$)	<u>2.68</u>	<u>3.52</u>	<u>2.72</u>	<u>3.64</u>	2.29	2.66	2.30	2.72
NextLat ($d=2$)	3.21	4.59	3.32	4.86	<u>2.38</u>	2.83	2.87	3.94

Table 7: Relative speedup and average accepted tokens per drafting steps over diverse domains. Note that “Accepted Tokens” excludes the next-token prediction which is always accepted.